

FibexSign

Documentación Técnica Completa

GrupoConex — Plataforma de firma electrónica self-hosted ("Firma")

Generado: 2026-09-24

Hecho por: Ciclo Fibex (GDA)

Fork interno mantenido de [OpenSign](#). Este documento consolida los 14 documentos técnicos de `docs/` en un único PDF de referencia.

Índice

1. Introducción y stack tecnológico
2. Variables de entorno
3. Configuración de Turborepo
4. Autenticación y multi-tenancy
5. Preparación y envío de documentos
6. Ceremonia de firma
7. Email Builder (plantillas de email)
8. OpenSignDrive (gestor de archivos)
9. Reportes (contactos, documentos, plantillas)
10. Internacionalización (i18n)
11. Arquitectura del frontend
12. Catálogo de Cloud Functions
13. Rate limiting
14. Guía de testing
15. Guía de despliegue

FibexSign

Plataforma de firma electrónica **self-hosted**, operada por **GrupoConex**. Es un fork de [OpenSign](#) mantenido internamente; en la interfaz de usuario el producto se presenta bajo el nombre **"Firma"**, mientras que `FibexSign` es el nombre del repositorio/proyecto interno.

Stack tecnológico

Monorepo **pnpm workspaces** + **Turborepo**, con dos aplicaciones:

- **apps/OpenSign** (paquete npm `open_sign`, frontend) — React 19 + Vite, Redux Toolkit, MUI + Radix Themes + DaisyUI + Tailwind (clases temáticas `op-*`), i18next (7 locales: `de`, `en`, `es`, `fr`, `hi`, `it`, `kr`), `pdf-lib` + `react-pdf` + `pkijjs` para manejo de PDF/certificados en cliente, `parse` (Parse JS SDK) para hablar con el backend, íconos vía `lucide-react`, tests con Vitest.
- **apps/OpenSignServer** (paquete npm `open_sign_server`, backend, ESM) — Express 5 + Parse Server 8 sobre MongoDB, adaptadores de storage S3/DigitalOcean Spaces o filesystem local, envío de mail vía Mailgun o SMTP, firma de documentos con certificado PFX/P12 (`@signpdf/*`), tests con Jasmine + `mongodb-runner`.
- **Orquestación**: Turborepo (`turbo.json`) coordina las tareas `dev`, `build` y `lint` de ambos paquetes.
- **Infraestructura local**: Docker Compose (`server`, `mongo`, `client`, `caddy` como reverse proxy TLS).

Requisitos de entorno

- Node.js 18, 20 o 22 (definido en `engines` de ambos `package.json`).
- [pnpm](#) 10.33.2 (fijado en `packageManager` del `package.json` raíz — usar Corepack o instalar esa versión exacta).
- Docker Desktop (o Docker Engine + Compose) para levantar MongoDB y, opcionalmente, el stack completo vía `docker compose`.

Quick start

```
# 1. Clonar el repo
git clone <url-del-repo> FibexSign
cd FibexSign

# 2. Instalar dependencias de todo el monorepo
pnpm install

# 3. Configurar variables de entorno
#   Desarrollo local rápido (valores no sensibles, ya versionados):
cp .env.local_dev .env
cp .env.local_dev apps/OpenSign/.env
#   Para producción o un entorno real, seguir docs/environment-variables.md
#   en vez de reusar .env.local_dev.

# 4. Levantar frontend + backend en paralelo (vía Turborepo)
pnpm dev
```

`pnpm dev` requiere MongoDB accesible en la URI configurada (`MONGODB_URI` / `DATABASE_URI`). Para levantar Mongo con Docker sin construir las imágenes de la app, ver el target `run` del `Makefile` (`docker compose up -d`); para un build completo del stack (frontend nativo + `docker compose up --build`), ver el target `build` del `Makefile`.

Comandos esenciales

Comando	Qué hace
<code>pnpm dev</code>	Corre <code>dev</code> de frontend y backend en paralelo vía Turborepo (<code>turbo dev --ui=tui</code>).
<code>pnpm dev:frontend</code>	Corre solo el frontend (<code>pnpm --filter open_sign dev</code> , Vite).

<code>pnpm dev:backend</code>	Corre solo el backend (<code>pnpm --filter open_sign_server dev</code> , <code>nodemon index.js</code>).
<code>pnpm build</code>	Build de producción de todos los paquetes (<code>turbo build</code>).
<code>pnpm lint</code>	Lint de todos los paquetes (<code>turbo lint</code>).
<code>pnpm --filter open_sign test</code>	Tests del frontend (Vitest).
<code>pnpm --filter open_sign_server test</code>	Tests del backend (<code>mongodb-runner start && TESTING=true jasmine</code> ; requiere Docker/Mongo disponible).

Variables de entorno

FibexSign usa **un único archivo** `.env.example` en la raíz del repositorio (no hay un `.env.example` por app dentro de `apps/OpenSign` o `apps/OpenSignServer`). El mismo patrón aplica a `.env.local_dev`, el archivo con valores de desarrollo local (no sensibles, versionado en git) que se usa para levantar el stack completo.

Cómo se distribuye el archivo `.env`

El `Makefile` de la raíz es quien copia el archivo a los lugares donde cada proceso lo necesita:

```
build:
  cp .env.local_dev .env
  cd apps/OpenSign && cp ../../.env.local_dev .env && npm install && npm run build
  HOST_URL=${HOST_URL} docker compose up --build --force-recreate

run:
  cp .env.local_dev .env
  docker compose up -d
```

Esto deja:

- `<raíz>/env` — usado por `docker-compose.yml` vía `env_file: .env` en los servicios `server` y `client`.
- `apps/OpenSign/.env` — usado por Vite (`loadEnv`) cuando el frontend se compila o corre nativamente (`npm run build` / `npm run dev`).

Importante — gap no cubierto por el `Makefile`: `apps/OpenSignServer` NO recibe una copia propia de `.env`. El backend carga variables con `dotenv.config()` (sin `path` explícito), lo que resuelve el archivo `.env` relativo al directorio de trabajo del proceso. Cuando el backend corre **dentro de Docker** (`make build` / `make run`), esto no es un problema porque `docker-compose.yml` inyecta las variables vía `env_file: .env` (raíz). Pero si se ejecuta el backend **fuera de Docker** con `pnpm dev:backend` / `pnpm --filter open_sign_server dev`, hay que copiar manualmente `.env` a `apps/OpenSignServer/.env` (o exportar las variables al shell), porque el `Makefile` no lo hace por vos.

Frontend config

Variable	Aplica a	Obligatoria	Valor por defecto	Descripción e impacto
<code>PUBLIC_URL</code>	Frontend	No (vestigial)	—	Comentario original: "Set it to the URL form where the app home page will be accessed". Es una variable heredada de Create React App. <code>vite.config.js</code> solo expone al bundle las variables con prefijo <code>REACT_APP_</code> ; <code>PUBLIC_URL</code> no lo tiene, y no aparece referenciada en <code>apps/OpenSign/src</code> ni en <code>server.cjs</code> . Actualmente no tiene ningún efecto en el build ni en el servidor de producción.
<code>GENERATE_SOURCEMAP</code>	Frontend	No (vestigial)	—	Comentario original: "Set it to true if you want to generate the Sourcemap for debugging". También heredada de CRA; Vite controla sourcemaps por su propia configuración de build (<code>build.sourcemap</code> en <code>vite.config.js</code> , no seteado hoy). No se encontró ninguna referencia a esta variable en el código actual.
<code>REACT_APP_SERVERURL</code>	Frontend	Recomendada	<code>window.location.origin + "/api/app"</code> (fallback en runtime)	Comentario original: "Set it to the URL from where APIs will be accessible...". Consumida en <code>apps/OpenSign/src/constant/appinfo.js</code> (<code>serverUrlFn</code>). Si no está definida, el frontend intenta construir la URL del API a partir del origen actual del navegador.
<code>REACT_APP_APPID</code>	Frontend	No	<code>opensign</code>	Comentario original: "A 12 character long random app identifier. The value of this should be same as APP_ID...". Debe coincidir con <code>APP_ID</code> del backend. Leída en <code>appinfo.js</code> con fallback a <code>'opensign'</code> .

REACT_APP_GTM	Frontend	No	—	Comentario original: "Google tag manager container id...". La dependencia <code>react-gtm-module</code> está declarada en <code>apps/OpenSign/package.json</code> , pero no se encontró ningún <code>import</code> de esa librería ni referencia a <code>REACT_APP_GTM</code> en <code>apps/OpenSign/src</code> . Es decir: está declarada y documentada, pero actualmente no está cableada a ningún componente (queda como reserva para integración futura).
---------------	----------	----	---	---

Backend ExpressJS config

Variable	Aplica a	Obligatoria	Valor por defecto	Descripción e impacto
APP_ID	Backend	No	opensign	Comentario original: "...should be same as REACT_APP_APPID...". Leída en <code>Utils.js</code> como <code>serverAppId = process.env.APP_ID 'opensign'</code> .
appName	Backend	No (ver discrepancia)	—	Comentario original: "Name of the app. It will be visible in the verification emails sent out.". Discrepancia verificada en código: <code>Utils.js</code> línea 22 define <code>export const appName = 'Firma';</code> como constante hardcodeada , ignorando por completo el valor de <code>process.env.appName</code> . Cambiar esta variable en <code>.env</code> no tiene ningún efecto hoy; el nombre visible en emails es siempre "Firma".
MASTER_KEY	Backend	Sí	—	Comentario original: "...secret key that allows access to all the data...". Se pasa directo a <code>masterKey</code> de Parse Server sin fallback. También se reinyecta como variable de entorno al subprocesso de migraciones (<code>node ./node_modules/parse-dbtool/...migrate</code>). Sin este valor, Parse Server arranca sin master key funcional.
MONGODB_URI	Backend	Condicional	—	Comentario original: "Mongodb URI to connect to". Ver discrepancia con DATABASE_URI en la sección de variables ausentes de <code>.env.example</code> más abajo — <code>DATABASE_URI</code> tiene prioridad si ambas están definidas.
PARSE_MOUNT	Backend	No	/app	Comentario original: "Path on which APIs should be mounted. Do not change this...". Leída en <code>index.js</code> como <code>mountPath</code> .
SERVER_URL	Backend	Recomendada	<code>http://localhost:\${PORT}/app</code> si <code>PORT</code> está seteado, si no <code>/app</code>	Comentario original: "...URL from where APIs will be accessible to the NodeJS functions...". Usada para <code>cloudServerUrl</code> (<code>Utils.js</code>) y <code>publicServerURL</code> de Parse Server (<code>index.js</code>). También se parsea con <code>new URL(...)</code> en el middleware de archivos firmados — si está mal formada, esa ruta lanza excepción.

Rate Limiting config

Variable	Aplica a	Obligatoria	Valor por defecto	Descripción e impacto
AUTH_RATE_LIMIT_WINDOW_MS	Backend	No	300000 (5 min)	Comentario original coincide con el código (<code>utils/authRateLimiter.js</code>). Ventana compartida por el limiter estricto (login) y el operacional.
AUTH_RATE_LIMIT_MAX	Backend	No	30	Comentario original coincide con el código. Aplica a rutas "estrictas" (<code>/login</code> , <code>/requestPasswordReset</code> , <code>/functions/loginuser</code> , <code>/functions/AuthLoginAsMail</code> , <code>/functions/SendOTPMailV1</code> , <code>/functions/addadmin</code> , <code>/functions/usersignup</code>).

OPERATIONAL_RATE_LIMIT_MAX	Backend	No	No es simplemente 300	Comentario original dice "(default: 300)", pero el código real es más específico: <code>Number(process.env.OPERATIONAL_RATE_LIMIT_MAX) (process.env.AUTH_RATE_LIMIT_MAX ? Number(process.env.AUTH_RATE_LIMIT_MAX) * 10 : 300)</code> . Es decir: si <code>OPERATIONAL_RATE_LIMIT_MAX</code> no está seteada pero <code>AUTH_RATE_LIMIT_MAX</code> sí, el default real es <code>AUTH_RATE_LIMIT_MAX × 10</code> , no <code>300</code> . Solo cae en <code>300</code> si ninguna de las dos está definida. Aplica a rutas operacionales (<code>/functions/getUserDetails</code> , <code>/functions/declinedoc</code>).
----------------------------	---------	----	-----------------------	--

Storage config (S3 / DigitalOcean Spaces)

Variable	Aplica a	Obligatoria	Valor por defecto	Descripción e impacto
USE_LOCAL	Backend	No	'false' (string) si no está seteada	Comentario original (citado tal cual, con el typo del archivo): " <i>Set this to true if you are using local storage instead of s3 compatible storage</i> ". Leída en <code>Utils.js</code> como <code>useLocal</code> , comparada en minúsculas. Si es distinto de <code>'true'</code> , el backend intenta usar <code>S3Adapter</code> .
DO_SPACE	Backend	Condicional	—	Nombre del bucket/space. Solo relevante si <code>USE_LOCAL !== 'true'</code> .
DO_ENDPOINT	Backend	Condicional	—	Si no incluye <code>http</code> , se le antepone <code>https://</code> automáticamente (<code>index.js</code>).
DO_BASEURL	Backend	Condicional	—	Base URL pública de los archivos.
DO_ACCESS_KEY_ID	Backend	Condicional	—	Credencial S3/DO.
DO_SECRET_ACCESS_KEY	Backend	Condicional	—	Credencial S3/DO.
DO_REGION	Backend	Condicional	<code>us-west</code> (valor de ejemplo, no hay fallback en código)	Región S3/DO.

Comportamiento de fallback verificado (no documentado en el comentario original): la construcción de `S3Adapter` está envuelta en `try/catch` en `index.js`. Si falla (credenciales inválidas, config incompleta), el backend **no aborta el arranque**: cae silenciosamente a `FSFilesAdapter` (almacenamiento local en `./files`) y solo deja un `console.log('Please provide AWS credentials in env file! Defaulting to local storage.')`. Esto significa que una mala configuración de storage en producción puede pasar desapercibida y el servidor terminar guardando archivos en disco local en vez del bucket esperado.

Email (Mailgun o SMTP)

Variable	Aplica a	Obligatoria	Valor por defecto	Descripción e impacto
MAILGUN_API_KEY	Backend	Condicional	—	Comentario original del bloque: " <i>(The app will not initialize if any of these 3 variables are not set)</i> ". Discrepancia verificada: esto no es exacto. El código solo evalúa <code>process.env.MAILGUN_API_KEY</code> como condición para intentar Mailgun (<code>else if (process.env.MAILGUN_API_KEY)</code>); <code>MAILGUN_DOMAIN</code> y <code>MAILGUN_SENDER</code> se leen sin validar que existan. Si faltan, Mailgun puede fallar en tiempo de envío, no en el arranque. Además, si ninguna de las 3 vars de Mailgun ni la config SMTP resultan válidas, la app igual arranca — simplemente no configura <code>emailAdapter</code> en Parse Server (bloque condicional <code>isMailAdapter === true</code>) y los emails no se envían.
MAILGUN_DOMAIN	Backend	Condicional	—	Ver nota anterior.
MAILGUN_SENDER	Backend	Condicional	—	Usado como remitente si SMTP no está habilitado (<code>mailsender</code>).

SMTP_ENABLE	Backend	No	false	Comparado en minúsculas contra 'true' (Utils.js , smtpenable). Si es true , se intenta SMTP antes que Mailgun.
SMTP_HOST	Backend	Condicional (si SMTP_ENABLE=true)	—	Host del transporte Nodemailer.
SMTP_PORT	Backend	No	465	Si no está seteado, index.js usa 465 . Además, smtpsecure (Utils.js) se calcula como true salvo que SMTP_PORT esté seteado y sea distinto de '465' — es decir, TLS implícito por defecto.
SMTP_USER_EMAIL	Backend	Condicional	—	Usado como remitente (mailsender) y como usuario de auth SMTP si SMTP_USERNAME (ver sección de variables ausentes) no está definida.
SMTP_PASS	Backend	Condicional	—	Password SMTP. Solo se agrega bloque auth al transporter si tanto usuario como password están presentes.

Certificado de firma (PFX/P12)

Variable	Aplica	Obligatoria	Valor por defecto	Descripción e impacto
PFX_BASE64	Backend	Sí (para firma con certificado)	—	Comentario original: "Base64 encoded PFX or p12 document signing certificate file". Su consumo concreto está en el código de cloud/ (flujo de firma con @signpdf/*), fuera de los archivos verificados para este documento (index.js , Utils.js , utils/*.js , auth/*.js).
PASS_PHRASE	Backend	Sí (si PFX_BASE64 está seteado)	—	Passphrase del certificado anterior.

Variables usadas en código pero ausentes de .env.example

Estas variables se leen directamente vía process.env en el backend (o, en un caso, en el frontend) pero **no aparecen documentadas en .env.example**. Cada una fue confirmada leyendo el código fuente real, no asumida.

Backend — del hallazgo original

Variable	Archivo	Comportamiento verificado
ACCOUNT_LOCKOUT_DURATION_MINUTES	index.js:111	Number(...) 5 . Minutos de bloqueo de cuenta tras superar el umbral de intentos fallidos (config nativa de Parse Server accountLockout.duration).
ACCOUNT_LOCKOUT_THRESHOLD	index.js:112	Number(...) 5 . Cantidad de intentos fallidos antes de bloquear la cuenta (accountLockout.threshold).
MASTER_KEY_IPS	index.js:106-110	Lista de IPs (separadas por coma, trimeadas, vacíos filtrados) autorizadas a usar el master key. Default confirmado si no está seteada: ['127.0.0.1', '::1'] — es decir, fuera de esas IPs el master key queda inutilizable aunque el valor sea correcto.
TRUST_PROXY_HOPS	index.js:114,186-193	Number(...) 0 . Configura app.set('trust proxy', N) de Express. Si es 0 (default), se hace app.set('trust proxy', false) y el propio código emite un console.warn explícito advirtiendo que, si el servidor corre detrás de un reverse proxy/load balancer en producción, el rate limiter de auth no podrá aislar atacantes de usuarios legítimos por IP real.
SESSION_LENGTH_SECONDS	index.js:113	Number(...) 60*60*24*30 (30 días). Duración de sesión de Parse Server (sessionLength).

DATABASE_URI	index.js:116-117	Tiene prioridad sobre <code>MONGODB_URI</code> : <code>process.env.DATABASE_URI</code> <code>process.env.MONGODB_URI</code> <code>'mongodb://localhost:27017/dev'</code> . Si ambas están seteadas con valores distintos, <code>DATABASE_URI</code> gana y <code>MONGODB_URI</code> se ignora silenciosamente — riesgo real de confusión en despliegues donde solo se documenta/configura <code>MONGODB_URI</code> .
GOOGLE_CLIENT_ID	index.js:177	Pasada como <code>auth.google.clientId</code> a Parse Server, para validar tokens de login social de Google en el backend.
PORT	index.js:262 (y <code>Utils.js:20</code> como fallback de <code>SERVER_URL</code>)	<code>process.env.PORT</code> <code>8080</code> . Puerto HTTP del backend.
SSO_API_URL	auth/authadapter.js:4,9	Base URL del proveedor SSO propio. El adapter custom de Parse (<code>SSOAuth.validateAuthData</code>) hace <code>GET {SSO_API_URL}/oauth/userinfo</code> con el <code>access_token</code> del usuario para validar el login SSO.
TESTING	index.js:242,261	Cualquier valor truthy desactiva el montaje de Parse Server y el <code>httpServer.listen(...)</code> , usado por la suite Jasmine (<code>TESTING=true jasmine</code>).

Backend — hallazgos adicionales (no listados en el issue original, confirmados durante esta revisión)

Variable	Archivo	Comportamiento verificado
CORS_ALLOWED_ORIGINS	utils/corsOptions.js	Lista de orígenes permitidos separados por coma. Si no está seteada, la lista queda vacía y se rechazan todas las requests cross-origin (<code>callback(null, false)</code>) para cualquier origen no vacío que no esté en la lista). El código emite un <code>console.warn</code> una sola vez advirtiendo que hay que setearla en producción. Esta es una variable crítica para que el frontend pueda hablar con el backend en producción y no aparece en absoluto en <code>.env.example</code> .
SMTP_USERNAME	index.js:75,80	Usuario de autenticación SMTP alternativo/explicito. Si está seteado, se usa en lugar de <code>SMTP_USER_EMAIL</code> como <code>auth.user</code> del transporter Nodemailer.

Frontend

Variable	Archivo	Comportamiento verificado
REACT_APP_GOOGLECLIENTID	apps/OpenSign/src/constant/appinfo.js:19-21	Client ID de Google OAuth para el botón de login social en el frontend (<code>appInfo.googleClientId</code>). Sin ella, el valor queda en cadena vacía <code>''</code> . No está documentada en <code>.env.example</code> pese a que su contraparte de backend (<code>GOOGLE_CLIENT_ID</code> , ver arriba) sí es una variable "conocida" del proyecto.

Frontend — variables de infraestructura del servidor de producción

`apps/OpenSign/server.cjs` (el servidor estático que sirve el build de producción, invocado por `npm start`) lee además:

Variable	Comportamiento verificado
PORT	<code>Number(process.env.PORT)</code> <code>3000</code> . Puerto donde escucha el servidor estático de producción del frontend.
HOST	<code>process.env.HOST</code> <code>"0.0.0.0"</code> . Interfaz de red donde escucha.

Ninguna de las dos aparece en `.env.example` ni en `.env.local_dev` .

Configuración de Turborepo y workspaces (pnpm)

FibexSign es un monorepo **pnpm workspaces** + **Turborepo**. Este documento explica cómo está configurado y cómo extenderlo.

pnpm-workspace.yaml

```
packages:
  - 'apps/*'
```

Cualquier directorio directo bajo `apps/` que contenga un `package.json` válido (con campo `name`) se registra automáticamente como paquete del workspace. pnpm resuelve el glob al hacer `pnpm install` y al ejecutar `pnpm --filter <nombre>`.

Nota verificada en el repo actual: bajo `apps/` también existen los directorios `localstack/` y `mongo/`, usados como utilidades de infraestructura local (script de LocalStack y un `Dockerfile` de Mongo respectivamente). Ninguno de los dos tiene `package.json`, por lo que **pnpm los ignora silenciosamente** como paquetes del workspace — no aparecen en `pnpm -r list` ni participan de las tareas de Turborepo. Esto es relevante para la sección "cómo agregar una tercera app" más abajo: el glob `apps/*` por sí solo no es suficiente, hace falta un `package.json`.

Los dos paquetes reales del workspace hoy son:

- `apps/OpenSign` → paquete npm `open_sign` (frontend).
- `apps/OpenSignServer` → paquete npm `open_sign_server` (backend).

turbo.json

```
{
  "$schema": "https://turbo.build/schema.json",
  "ui": "tui",
  "tasks": {
    "dev": {
      "persistent": true,
      "cache": false
    },
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["build/**", "dist/**"]
    },
    "lint": {
      "dependsOn": ["^lint"]
    }
  }
}
```

- **ui: "tui"** — Turborepo renderiza su interfaz de terminal interactiva (paneles por paquete) en vez del log lineal plano.
- **Tarea dev**
 - `persistent: true` : le dice a Turborepo que este proceso no termina solo (es un watcher/servidor de larga duración: `vite` para el frontend, `nodemon` para el backend). Turborepo no espera a que "termine" para considerar la tarea completa, y la mantiene corriendo en su panel de la TUI.
 - `cache: false` : nunca cachea resultados de `dev` (no tendría sentido cachear un servidor persistente).
 - Se dispara con `pnpm dev → turbo dev --ui=tui`, que corre la tarea `dev` de **todos** los paquetes del workspace que la definan, en paralelo, respetando el grafo de dependencias entre paquetes si existiera alguno (hoy `open_sign` y `open_sign_server` no dependen entre sí en sus respectivos `package.json`, así que corren simplemente en paralelo).
- **Tarea build**
 - `dependsOn: ["^build"]` : el prefijo `^` es la sintaxis de Turborepo para "las dependencias de este paquete dentro del workspace, no el paquete mismo". Es decir: *antes* de correr `build` en un paquete dado, Turborepo primero debe correr (y completar con éxito) la tarea `build` de todo paquete del workspace del que ese paquete dependa (vía `dependencies` / `devDependencies` internas al monorepo). Con la topología actual (sin dependencias cruzadas entre `open_sign` y `open_sign_server`), esta regla no tiene efecto práctico hoy — pero es la que garantiza el orden correcto automáticamente el día que, por ejemplo, se extraiga un paquete compartido `apps/shared-types` del que ambos dependan.
 - `outputs: ["build/**", "dist/**"]` : le dice a Turborepo qué carpetas cachear/restaurar por paquete. `apps/OpenSign` compila a `build/` (`vite build`, ver `outDir: "build"` en `vite.config.js`); `dist/**` queda declarado de forma genérica para cubrir cualquier paquete futuro que compile ahí (`open_sign_server` no tiene paso de build — corre directo con `node index.js` — por lo que hoy no genera ni `build/` ni `dist/`).
 - Se dispara con `pnpm build → turbo build`.
- **Tarea lint**
 - `dependsOn: ["^lint"]` : mismo mecanismo que `build`, aplicado a `lint` — lintea primero las dependencias internas del workspace antes que el paquete dependiente. Sin dependencias cruzadas hoy, corre lint de ambos paquetes en el orden que Turborepo determine, cacheando el resultado si el código no cambió (a diferencia de `dev`, `lint` sí puede cachear porque no define `cache: false`).
 - Se dispara con `pnpm lint → turbo lint`.

Cómo los scripts raíz mapean a `pnpm --filter`

El `package.json` de la raíz no tiene dependencias propias — es puramente un orquestador:

Script raíz	Qué hace
<code>pnpm dev</code>	<code>turbo dev --ui=tui</code> — corre <code>dev</code> en todos los paquetes del workspace en paralelo, vía el grafo de Turborepo.
<code>pnpm dev:frontend</code>	<code>pnpm --filter open_sign dev</code> — bypassa Turborepo por completo y ejecuta el script <code>dev</code> solo del paquete <code>open_sign</code> (<code>apps/OpenSign</code>), como si hicieras <code>cd apps/OpenSign && pnpm dev</code> .
<code>pnpm dev:backend</code>	<code>pnpm --filter open_sign_server dev</code> — mismo mecanismo, pero apunta al paquete <code>open_sign_server</code> (<code>apps/OpenSignServer</code>).
<code>pnpm build</code>	<code>turbo build</code> — build de todos los paquetes con caché y orden de dependencias (<code>^build</code>).
<code>pnpm lint</code>	<code>turbo lint</code> — lint de todos los paquetes con caché y orden de dependencias (<code>^lint</code>).

`pnpm --filter <nombre>` usa el campo `name` del `package.json` de cada paquete (`open_sign`, `open_sign_server`), **no** el nombre del directorio (`OpenSign`, `OpenSignServer`) — son distintos y hay que usar el del `package.json`.

Cómo agregar una tercera app al monorepo

Pasos verificados contra el comportamiento real de `pnpm` + Turborepo en este repo (corrige la asunción de que "solo crear el directorio alcanza"):

1. Crear el directorio bajo `apps/` (ej. `apps/mi-nueva-app/`).
2. Agregar un `package.json` mínimo con un campo `name` único dentro del workspace (ej. `"name": "mi_nueva_app"`). **Sin este archivo, pnpm ignora el directorio** — es exactamente lo que pasa hoy con `apps/localstack` y `apps/mongo`, que no tienen `package.json` y por eso no son paquetes del workspace pese a matchear el glob `apps/*`.
3. Correr `pnpm install` desde la raíz para que pnpm registre el nuevo paquete y linkee su `node_modules`.
4. Si la nueva app define scripts `dev`, `build` y/o `lint` en su propio `package.json`, Turborepo los recoge automáticamente la próxima vez que corras `pnpm dev` / `pnpm build` / `pnpm lint` — **no hace falta tocar `turbo.json` ni `pnpm-workspace.yaml`**, porque las tareas están definidas de forma genérica por nombre de script, no por paquete explícito.
5. Si la nueva app va a depender de otro paquete del workspace (ej. un paquete de tipos compartidos), agregar esa dependencia en su `package.json` como `"mi-paquete-compartido": "workspace:*` — recién ahí `dependsOn: ["^build"]` empieza a tener efecto real para esa app.
6. Opcional: agregar un script de conveniencia en el `package.json` raíz (ej. `"dev:mi-nueva-app": "pnpm --filter mi_nueva_app dev"`), siguiendo el mismo patrón que `dev:frontend` / `dev:backend`.

Autenticación y multitenancy

Este documento describe cómo FibexSign autentica usuarios (con y sin contraseña), cómo se administra la sesión sobre el SDK de Parse, cómo funciona el aislamiento multi-tenant (`partners_Tenant`) y cómo se modela el perfil de usuario extendido (`contracts_Users`). Todo lo aquí descrito fue verificado leyendo el código fuente en `apps/OpenSignServer/cloud/parsefunction/` y `apps/OpenSign/src/` ; donde el issue original asumía un mecanismo que no corresponde al código real, se indica explícitamente.

Nota sobre `withSessionValidation`

El pedido original de documentación asumía la existencia de un middleware de backend llamado `withSessionValidation` . Verificación real:

- **No existe** ningún middleware ni función con ese nombre en `apps/OpenSignServer` . Cada Cloud Function valida la sesión por su cuenta (ver más abajo).
- **Sí existe** `withSessionValidation` como utilidad de **frontend**, en `apps/OpenSign/src/utils/withSessionValidation.js` . No es un middleware de servidor: es un higher-order function que envuelve *handlers* de UI (más de 20 componentes lo usan) para comprobar, antes de ejecutar la acción, que exista `localStorage.TenantId` y un `Parse.User.current().getSessionToken()` válido. Si falta alguno, despacha `sessionStatus(false)` a Redux (`apps/OpenSign/src/redux/reducers/userReducer.js`) y aborta sin llamar al backend.

```
// apps/OpenSign/src/utils/withSessionValidation.js
export function withSessionValidation(fn) {
  return async (...args) => {
    const tenantId = localStorage.getItem("TenantId");
    const sessionToken = Parse.User.current()?.getSessionToken?();
    if (!tenantId || !sessionToken) {
      store.dispatch(sessionStatus(false));
      throw new Error("invalid session token");
    }
    return await fn(...args);
  };
}
```

Cuando `sessionStatus` pasa a `false` , `HomeLayout` (`apps/OpenSign/src/layout/HomeLayout.jsx`) y la ruta `Validate` (`apps/OpenSign/src/primitives/Validate.jsx`) dejan de renderizar el `Outlet` y muestran `SessionExpiredModal` , que fuerza `Parse.User.logout()` y redirige a login. Es, en definitiva, un guard de UX en el cliente — **no** reemplaza ni implementa control de acceso en el servidor.

La autorización real de cada operación ocurre en el backend, función por función, como se describe a continuación.

Sesión: Parse SDK, sin middleware centralizado

Parse Server gestiona las sesiones de forma nativa: `Parse.User.login()` / `Parse.User.become(sessionToken)` en el cliente producen un `sessionToken` que viaja en el header `X-Parse-Session-Token` en cada request subsiguiente del SDK. Parse Server resuelve ese token y, si es válido, puebla `request.user` antes de invocar la Cloud Function.

No hay ningún middleware centralizado que rechace requests sin sesión. El patrón real, repetido de forma manual en decenas de funciones, es:

```
// apps/OpenSignServer/cloud/parsefunction/addUser.js
if (!request.user) {
  throw new Parse.Error(Parse.Error.INVALID_SESSION_TOKEN, 'Invalid session token.');
```

Ejemplos verificados de este patrón: `addUser.js` , `getTenant.js` , `updateTenant.js` , `getUserListByOrg.js` , `filterDocs.js` , `getSigners.js` , `updatesignaturetype.js` . Cada función es responsable de su propio chequeo; una función nueva que olvide el `if (!request.user)` quedaría

abierta sin que ningún otro componente lo detecte. Esto es una observación de diseño, no necesariamente un defecto — es coherente con cómo Parse Cloud Code se usa habitualmente — pero implica que la superficie de auditoría de seguridad es "una función a la vez".

Algunas funciones (`declinedocument.js` , `cloud/parsefunction/pdf/PDF.js`) sólo exigen `request.user` **condicionalmente**, cuando el documento tiene `IsEnableOTP: true` ; si el flag está apagado, aceptan el `userId` como parámetro sin sesión Parse asociada (ver `docs/signing-ceremony.md`).

Tampoco existe `Parse.Role` / la clase `_Role` de Parse en este código — no hay un sistema de roles formal. Los "roles" (`contracts_Admin` , `contracts_OrgAdmin` , `contracts_Guest`) son valores de texto libre en el campo `UserRole` de `contracts_Users` / `contracts_Contactbook` , comparados manualmente:

```
// apps/OpenSignServer/cloud/parsefunction/updateTenant.js
const callerRole = callerExtUser.get('UserRole');
if (callerRole !== 'contracts_Admin' && callerRole !== 'contracts_OrgAdmin') {
  throw new Parse.Error(Parse.Error.OPERATION_FORBIDDEN, 'Unauthorized.');
```

De nuevo: observación de arquitectura, no un hallazgo de seguridad puntual — pero documentarlo es importante porque un futuro rol nuevo requiere tocar cada función relevante a mano, y no hay un catálogo único de permisos.

Login con contraseña

`apps/OpenSign/src/pages/Login.jsx` invoca el flujo estándar de Parse (`Parse.User.login` vía la Cloud Function `loginuser` → `apps/OpenSignServer/cloud/parsefunction/loginUser.js`):

```
// loginUser.js
const user = await Parse.User.login(username, password);
```

Errores de Parse (`OBJECT_NOT_FOUND` , `CONNECTION_FAILED` , rate limit 429) se traducen en `Login.jsx` a mensajes localizados (`invalid-username-password-region` , `too-many-login-attempts` , `server-error`). Tras el login, `Login.jsx` resuelve el perfil extendido (`contracts_Users`) para decidir a qué ruta redirigir según `UserRole` ; si no encuentra el registro extendido, cierra la sesión (`logoutUser()`).

Login sin contraseña (OTP) — firmante invitado y verificación de email

Este flujo es el mecanismo real detrás de "login sin contraseña": un código OTP de un solo uso, no un login social ni un magic-link JWT.

- Solicitud del código** — `Parse.Cloud.run('SendOTPMailV1', { email, docId })` invoca `apps/OpenSignServer/cloud/parsefunction/SendMailOTPv1.js` . Genera un código numérico de 4 dígitos (`randomInt(1000, 10000)`), lo envía por email vía `Parse.Cloud.sendEmail` , y lo persiste en la clase Parse `defaultdata_Otp` (campos `Email` , `OTP` , `ExpiresAt` = +10 minutos, `FailedAttempts`: 0). Si el pedido de OTP viene ligado a un documento (`docId`), además actualiza el contador de correos del `ExtUserPtr` dueño del documento.
- Verificación del código** — el cliente llama directamente por REST (no `Parse.Cloud.run`) a `functions/AuthLoginAsMail` (`apps/OpenSignServer/cloud/parsefunction/AuthLoginAsMail.js`). Esta función:
 - Busca el registro en `defaultdata_Otp` por `Email` .
 - Rechaza con `'Invalid Otp'` si expiró (`ExpiresAt`), si superó `MAX_OTP_ATTEMPTS` = 5 intentos fallidos, o si el código no coincide — incrementando `FailedAttempts` en cada intento fallido.
 - Si el código es válido, resuelve el `Parse.User` cuyo `email` coincide y llama internamente al endpoint `POST {serverUrl}/loginAs` **usando la Master Key**, que devuelve un `sessionToken` real de Parse para ese usuario sin haber usado contraseña.
 - Si el usuario tenía `emailVerified: false` , lo marca en `true` en el mismo paso (por eso esta función también sirve como verificación de email).
- Establecimiento de sesión en el cliente** — `GuestLogin.jsx` recibe { `sessionToken` , `...user` } y llama `await Parse.User.become(_user.sessionToken)` , guarda `accesstoken` , `UserInfo` y la clave `Parse/<appId>/currentUser` en `localStorage` , y navega a `/load/recipientSignPdf/:docId/:contactId` . Desde ese punto, el firmante invitado tiene una sesión Parse legítima igual que un usuario logueado con contraseña.

`GuestLogin.jsx` es el punto de entrada para firmantes **sin cuenta previa** en la plataforma: si el email no corresponde a ningún contacto conocido del documento, el propio componente ofrece un formulario (`handleUserData` → `Parse.Cloud.run('linkcontacttodoc', ...)` → `apps/OpenSignServer/cloud/parsefunction/linkContactToDoc.js`) que crea el contacto (`contracts_Contactbook` , `UserRole: 'contracts_Guest'`) y lo vincula al documento antes de continuar.

Diagrama — login con contraseña vs. login OTP (firmante invitado)

```
sequenceDiagram
    actor U as Usuario / Firmante
    participant FE as Frontend (Login.jsx / GuestLogin.jsx)
    participant CC as Cloud Code
    participant DB as MongoDB (Parse classes)

    alt Login con contraseña
        U->>FE: email + password
        FE->>CC: Cloud Function "loginuser"
        CC->>CC: Parse.User.logIn(username, password)
        CC-->>FE: User + sessionToken (Parse nativo)
    else Login OTP (firmante invitado)
        U->>FE: ingresa email
        FE->>CC: Cloud Function "SendOTPMailV1" {email, docId}
        CC->>DB: upsert defaultdata_Otp {OTP, ExpiresAt, FailedAttempts:0}
        CC-->>U: email con código OTP
        U->>FE: ingresa código OTP
        FE->>CC: POST /functions/AuthLoginAsMail {email, otp}
        CC->>DB: lee defaultdata_Otp por Email
        alt OTP válido y no expirado y no bloqueado
            CC->>CC: POST interno /loginAs (Master Key) por userId
            CC-->>FE: { sessionToken, ...user }
            FE->>FE: Parse.User.become(sessionToken)
        else OTP inválido / expirado / bloqueado
            CC->>DB: FailedAttempts += 1
            CC-->>FE: "Invalid Otp"
        end
    end
    end
    FE->>FE: guarda accesstoken/UserInformation en localStorage
    Note over FE,CC: cada Cloud Function siguiente valida<br/>request.user manualmente (sin middleware central)
```

Multitenancy: `partners_Tenant`

El aislamiento entre organizaciones se modela con la clase Parse `partners_Tenant` , no con bases de datos ni esquemas separados:

- `apps/OpenSignServer/cloud/parsefunction/getTenant.js` (`gettenant`) resuelve el tenant a partir de `userId` (requiere `request.user`) o de `contactId` (sin requerir sesión, para flujos de firmante invitado).
- `apps/OpenSignServer/cloud/parsefunction/updateTenant.js` (`updatetenant`) permite actualizar campos de branding/plantillas de correo (`CompletionBody` , `CompletionSubject` , `RequestBody` , `RequestSubject` , `EmailEditorType`) — pero **deriva el tenant y el rol del llamante desde el servidor** (consulta `contracts_Users` por `request.user.id`) en vez de confiar en el `tenantId` recibido del cliente, y sólo permite `contracts_Admin` / `contracts_OrgAdmin` cuyo `TenantId` coincide con el tenant que intentan modificar.
- `apps/OpenSignServer/cloud/parsefunction/TenantAfterFind.js` es un hook `afterFind` sobre `partners_Tenant` que resuelve URLs firmadas (presigned) para `Logo` / `Favicon` antes de devolver el objeto al cliente.
- `apps/OpenSignServer/cloud/parsefunction/GetLogoByDomain.js` (`getlogobydomain`) resuelve branding **por dominio**, sin sesión: consulta `partners_Tenant` por `Domain` y devuelve `{ logo, favicon, apname, user: 'exist' | 'not_exist' }` . Se usa para pintar el logo correcto en la pantalla de login antes de que el usuario se identifique, y para distinguir instalaciones "greenfield" (sin ningún tenant) de instalaciones ya provisionadas.

No hay row-level security de Mongo ni particionado físico: el aislamiento depende de que cada consulta relevante incluya un filtro por `TenantId` / `OrganizationId` a nivel de Cloud Function (ver también `getUserListByOrg.js` , que valida explícitamente que el `OrganizationId` pedido pertenezca al tenant/organización del llamante antes de listar usuarios).

Perfil de usuario extendido: `contracts_Users` (no "UserExtend")

El issue original se refería a esto informalmente como "UserExtend". **En el código no existe una clase con ese nombre**; la clase real es `contracts_Users`, un objeto Parse independiente con un puntero (`UserId`) a `_User`, que guarda los datos que Parse `_User` no modela (rol, tenant, organización, plan de firma, preferencias, timezone, etc.).

- `apps/OpenSignServer/cloud/parsefunction/isextenduser.js` (`isextenduser`) sólo comprueba, por email, si ya existe un registro `contracts_Users` — se usa para decidir si un usuario recién autenticado necesita completar el onboarding.
- `apps/OpenSignServer/cloud/parsefunction/getUserDetails.js` resuelve el registro `contracts_Users` del usuario actual (o por email), incluyendo `TenantId` y excluyendo campos sensibles (`google_refresh_token`, `TenantId.PfxFile`, `authData`).
- En el frontend, el registro se cachea en `localStorage` bajo las claves `Extend_Class` (array con el objeto `contracts_Users`) y `UserInfo` (objeto `_User` crudo de Parse). `apps/OpenSign/src/pages/UserProfile.jsx` lee ambas para pintar/editar el perfil, y envuelve la mutación (`updateExtUser`) con `withSessionValidation` antes de llamar al backend — el mismo guard de UI descrito arriba, no una autorización de servidor adicional.
- `updatesignaturetype.js`, `updatePreferences.js` y otras funciones de preferencias actualizan el mismo objeto `contracts_Users` (localizándolo vía `request.user.id`), nunca el objeto `_User` de Parse directamente.

Preparación y envío de documentos

Este documento cubre el flujo de negocio desde que un usuario sube o crea un PDF hasta que el documento queda enviado a sus destinatarios: renderizado del PDF en el editor, tipos de widget disponibles, asignación de destinatarios y el orden (real, verificado en código) en que deben firmar. Todo lo descrito está verificado contra `apps/OpenSign/src/components/pdf/` y `apps/OpenSignServer/cloud/parsefunction/`.

Subida y renderizado del PDF

`apps/OpenSign/src/components/pdf/RenderAllPdfPage.jsx` es el punto de entrada de subida de archivo (`handleFileUpload`):

1. Valida tipo (`application/pdf`) y tamaño máximo configurado (`maxFileSize`).
2. Convierte el archivo a `ArrayBuffer` y verifica si está protegido con contraseña (`isPdfPasswordProtected`).
3. Ejecuta `clearAcroFields` (limpieza best-effort de campos de formulario/AcroForm preexistentes en el PDF, para evitar datos residuales de un PDF reciclado).
4. Si el PDF está cifrado, intenta desencriptarlo (`decryptPdf`) y, si falla sin contraseña, pide una contraseña al usuario (`prompt`) antes de reintentar.

El renderizado página a página ocurre en `apps/OpenSign/src/components/pdf/RenderPdf.jsx`, sobre `react-pdf / pdfjs-dist` (worker cargado desde CDN en `App.jsx : pdfjs.GlobalWorkerOptions.workerSrc`). Este componente:

- Mantiene refs por página (`pageRefs`) para scroll-tracking y para resolver sobre qué página cae un drop de widget.
- Calcula el escalado real de cada widget respecto al tamaño original del PDF vs. el tamaño renderizado en pantalla (`getContainerScale`, `posWidth`, `posHeight`, usando `defaultWidthHeight(pos.type)` como tamaño por defecto según el tipo de widget).
- Soporta pinch-to-zoom (`usePdfPinchZoom`) y guías de alineación (`useGuidelinesContext`).

`apps/OpenSign/src/components/pdf/EditTemplate.jsx` es el modal donde se configuran los metadatos del documento/plantilla antes o durante la edición: nombre, nota, descripción (con límites `MAX_NAME_LENGTH / MAX_NOTE_LENGTH / MAX_DESCRIPTION_LENGTH` reforzados también server-side en `DocumentBeforeSave.js`), recordatorios automáticos, redirect URL post-firma, colores de lapicera, y — relevante para este documento — los flags de orden de firma y OTP que se detallan más abajo. También calcula un hash de metadatos de páginas (`getPdfMetadataHash`, SHA-256 sobre ancho/alto de cada página) usado para detectar si el PDF subido cambió de estructura al reemplazar el archivo de una plantilla existente.

Tipos de widget

`apps/OpenSign/src/components/pdf/WidgetComponent.jsx` define el catálogo real de widgets arrastrables (`draggableItems`):

```
signature, stamp, initials, textInputWidget, name, job title, company,
email, date, textWidget, cellsWidget, checkbox, dropdown, radioButtonWidget,
image, drawWidget
```

Cada entrada se convierte en un elemento arrastrable vía `useWidgetDrag` (hook de drag-and-drop propio, en `apps/OpenSign/src/hook/useWidgetDrag.js`) y se renderiza con `apps/OpenSign/src/components/pdf/getWidgetType.jsx`, que soporta una variante compacta para mobile (`isMobileView`). `apps/OpenSign/src/components/pdf/WidgetList.jsx` es quien itera la lista de widgets ya colocados (`props.updateWidgets()`) y engancha los refs de drag sólo en desktop.

Al soltar un widget sobre el PDF, `RenderPdf.jsx` calcula la posición absoluta corrigiendo por el `containerScale` de la página, y el tamaño por defecto proviene de `defaultWidthHeight(pos.type)` (`apps/OpenSign/src/constant/Utils.js`) salvo que el widget ya tenga `Width / Height` explícitos guardados.

Asignación de destinatarios

- `apps/OpenSign/src/components/pdf/RecipientList.jsx` es la lista de destinatarios ya agregados al documento, con drag handles (`e.dataTransfer.setData`) para reordenar manualmente, color por firmante (`nameColor` , `darkenColor`) e indicador de si el firmante ya tiene al menos un widget colocado (`isWidgetExist`).
- `apps/OpenSign/src/components/pdf/SignerListPlace.jsx` envuelve `RecipientList` en las pantallas de colocación de campos (`PlaceholderSign.jsx` , `TemplatePlaceholder.jsx`) y expone `handleAddRecipient` para agregar un firmante nuevo desde ese mismo panel.
- `apps/OpenSign/src/components/shared/fields/SignersInput.jsx` es el selector usado en el formulario de creación/edición (`EditTemplate.jsx` , `Form.jsx`): un multi-select con función `arrayMove` para **reordenar destinatarios por drag** (`onSortEnd`). Este orden visual es el que alimenta el array `Signers / Placeholders` que se persiste en `contracts_Document`.
- La búsqueda de contactos existentes al tipear un email/nombre usa la Cloud Function `getsigners` → `apps/OpenSignServer/cloud/parsefunction/getSigners.js`, que busca en `contracts_Contactbook` por `Name / Email` (regex escapado) filtrando por `CreatedBy` del usuario autenticado — requiere `request.user`.

Orden de firma: `SendinOrder` y `SendInOrderStrict` (verificado)

El issue pedía verificar si existe un orden real de firmantes en vez de asumirlo. Se confirmó que sí existe, con dos flags independientes sobre `contracts_Document / contracts_Template`:

- **`SendinOrder`** (boolean): indica que el documento tiene un orden de firmantes definido (el orden del array `Signers / Placeholders` tal como quedó tras el drag-reorder en `SignersInput`). Se persiste en `EditTemplate.jsx` (`formData.SendinOrder`) y se guarda en `createDocumentFromApp.js`, `createBatchDocs.js` y `saveAsTemplate.js`.
- **`SendInOrderStrict`** (boolean, agregado en la migración `20260430000000-add_sendinorderstrict_field.cjs`): habilita la **aplicación forzosa** del orden en el momento de firmar.

La aplicación real del orden ocurre del lado servidor, en la Cloud Function de firma (`apps/OpenSignServer/cloud/parsefunction/pdf/PDF.js`, función `PDF / signPdf`), **no** en el frontend:

```
// apps/OpenSignServer/cloud/parsefunction/pdf/PDF.js
if (reqUserId && _resDoc?.SendinOrder === true && _resDoc?.SendInOrderStrict === true) {
  const placeholders = _resDoc.Placeholders.filter(p => p?.Role !== 'prefill');
  const myIdx = findPlaceholderIndex(placeholders, reqUserId);
  if (myIdx > 0) {
    const pendingId = findPendingPriorSigner(placeholders, myIdx, _resDoc?.AuditTrail);
    if (pendingId) {
      throw new Parse.Error(Parse.Error.OPERATION_FORBIDDEN,
        'Strict signing order is enabled – please wait for the previous signers to complete their action before signing.');
```

Es decir: si `SendinOrder` está activo pero `SendInOrderStrict` no, el orden es sólo una sugerencia de UI/notificación (por ejemplo, para decidir a quién notificar primero); si ambos están activos, el backend rechaza con `OPERATION_FORBIDDEN` cualquier intento de firma fuera de secuencia, comparando el índice del firmante actual contra las entradas de `AuditTrail` de los firmantes anteriores. El chequeo se salta para el dueño del documento (`className === 'contracts_Users'`, es decir cuando no hay `reqUserId`), porque el dueño no firma por este camino.

Creación y envío del documento

Dos caminos, según si se envía a uno o a muchos documentos/destinatarios a la vez:

- **createdocumentfromapp** → `apps/OpenSignServer/cloud/parsefunction/createDocumentFromApp.js` : crea un único `contracts_Document` a partir del payload armado en el editor (requiere `request.user`). Persiste, entre otros, `Name`, `URL`, `ExtUserPtr`, `CreatedBy`, `OriginIp`, `SentToOthers`, `SendinOrder`, `SendInOrderStrict`, `IsEnableOTP`, `AllowModifications`, `AutomaticReminders`, `NotifyOnSignatures`, `TimeToCompleteDays`, `RemindOnceInEvery`, `Signers[]`, `Placeholders[]`, `SignatureType[]`, `Bcc[]`, `Cc[]`, `PenColors[]`. Al terminar, actualiza el contador de documentos del `ExtUserPtr` (`setDocumentCount`).
- **batchdocuments** → `apps/OpenSignServer/cloud/parsefunction/createBatchDocs.js` : cubre el envío masivo ("bulk send"/"quick send"), con control de concurrencia manual (`chunkArray` + `mapWithConcurrency`, un pool de workers async) para no saturar la API al crear/enviar muchos documentos a la vez. Envía un correo resumen al dueño (`sendOwnerSummaryEmail`) y, por cada destinatario, un correo individual con el link de firma (`login/<base64(docId/email)>`), reemplazando variables de plantilla (`replaceMailVariables`) o cayendo al cuerpo por defecto (`mailTemplate`). Descuenta el conteo de documentos (`deductcount`) según la cantidad de documentos realmente creados.

Los documentos también pueden buscarse/filtrarse por nombre desde el dashboard vía `filterdocs` → `apps/OpenSignServer/cloud/parsefunction/filterDocs.js` (`fetchDocumentsByName`), que exige `request.user`, escapa el término de búsqueda antes de armar la regex, y siempre filtra por `CreatedBy` igual al usuario autenticado — nunca expone documentos de otro usuario por este camino.

Hooks sobre `contracts_Document`

Registrados en `apps/OpenSignServer/cloud/main.js`:

- **beforeSave** (`DocumentBeforeSave.js`): en creación, valida longitudes máximas de `Name` / `Note` / `Description` y que la combinación de `TimeToCompleteDays` / `RemindOnceInEvery` no genere más de 15 recordatorios. En updates, detecta la transición a "documento enviado" (aparición de `SignedUrl`) para descontar el conteo de documentos del dueño y setear `DocSentAt`.
- **afterSave** (`DocumentAfterSave.js`): reconstruye el **ACL** del objeto según tenga o no firmantes — `updateAclDoc` da lectura/escritura a `CreatedBy` y a cada firmante (`Signers[]`.`UserId` o, si son firmantes externos vía `contracts_Users`, sus punteros `ExtUserPtr`); `updateSelfDoc` restringe el acceso únicamente al creador cuando el documento no tiene firmantes (caso de auto-firma). El ACL nunca deja lectura/escritura pública.
- **afterFind** (`DocumentAfterFind.js`): resuelve URLs firmadas (presigned) para `SignedUrl`, `URL` y `CertificateUrl` antes de devolver el objeto (o rutas locales si `useLocal === 'true'`), y resuelve imágenes de prefill (`handleValidImage`) cuando el documento tiene placeholders con `Role === 'prefill'`.

Diagrama — de la subida del PDF al envío a firmar

```

flowchart TD
    A[Usuario sube PDF<br/>RenderAllPdfPage.jsx] --> B{Cifrado o<br/>con AcroFields?}
    B -->|sí| C[decryptPdf / clearAcroFields]
    B -->|no| D[RenderPdf.jsx renderiza páginas]
    C --> D
    D --> E[Arrastra widgets<br/>WidgetComponent / WidgetList]
    E --> F[Asigna destinatarios<br/>SignersInput / RecipientList]
    F --> G[Configura EditTemplate.jsx<br/>SendinOrder, SendInOrderStrict, IsEnableOTP]
    G --> H{Envío masivo?}
    H -->|no| I[Cloud Function<br/>createdocumentfromapp]
    H -->|sí| J[Cloud Function<br/>batchdocuments]
    I --> K[(contracts_Document)]
    J --> K
    K --> L[beforeSave: valida longitudes,<br/>marca DocSentAt]
    L --> M[afterSave: recalcula ACL<br/>updateAclDoc / updateSelfDoc]
    M --> N[Email a cada firmante<br/>con link de firma]
    N --> O[Firmante abre el link]
    O --> P{SendinOrder &&<br/>SendInOrderStrict?}
    P -->|sí, firmante previo pendiente| Q[Cloud Function signPdf<br/>rechaza OPERATION_FORBIDDEN]
    P -->|no o ya le toca| R[Continúa ceremonia de firma]
  
```

Ceremonia de firma

Este documento cubre el flujo del lado del firmante: acceso como firmante invitado (sin cuenta previa), verificación de identidad por OTP, generación de la firma (trazo, texto/imagen), embebido de la firma en el PDF final, generación del certificado de finalización/audit trail y el rechazo (decline) de un documento. Verificado contra `apps/OpenSign/src/pages/GuestLogin.jsx`, `apps/OpenSign/src/pages/SignyourselfPdf.jsx`, `apps/OpenSign/src/components/pdf/tab/`, y las Cloud Functions bajo `apps/OpenSignServer/cloud/parsefunction/`.

Acceso del firmante invitado

Un firmante recibe un link de firma (generado en `createBatchDocs.js` / `createDocumentFromApp.js` con forma `login/<base64(docId/email)>` o `recipientSignPdf/:docId/:contactId`) y llega a `apps/OpenSign/src/pages/GuestLogin.jsx`, que **no requiere cuenta previa en la plataforma**:

1. Si el email ya corresponde a un contacto conocido del documento, el firmante sólo necesita confirmar su email y pedir un OTP (`Parse.Cloud.run('SendOTPMailV1', { email, docId })`).
2. Si el email es nuevo, `GuestLogin.jsx` pide datos básicos (`handleUserData`) y llama `linkcontacttodoc` (`apps/OpenSignServer/cloud/parsefunction/linkContactToDoc.js`), que crea el contacto en `contracts_Contactbook` con `UserRole: 'contracts_Guest'` (un valor de texto, no un `Parse.Role`) y lo vincula al documento (placeholder, `Signers`, `ACL`).
3. Verificado el OTP (ver `docs/authentication-and-multitenancy.md` para el detalle del intercambio con `AuthLoginAsMail`), el cliente ejecuta `Parse.User.become(sessionToken)`: **el firmante invitado termina con una sesión Parse real**, no con un token temporal aparte. A partir de aquí, todas las llamadas del firmante llevan `X-Parse-Session-Token` como cualquier usuario logueado.
4. El cliente navega a `/load/recipientSignPdf/:docId/:contactId` (o `/recipientSignPdf/:docId/:contactId`), rutas envueltas por el componente `Validate` (`apps/OpenSign/src/primitives/Validate.jsx`), que re-verifica el `sessionToken` guardado contra `Parse.Query(Parse.User).get(...)` antes de mostrar el contenido; si falla, renderiza `SessionExpiredModal`.

Verificación de identidad: el OTP es por documento, no global

`IsEnableOTP` es un flag de `contracts_Document` / `contracts_Template`, configurado por el remitente al preparar el documento (ver `docs/document-preparation-and-sending.md`). Su efecto se ve en dos puntos del backend:

- **Firma** (`apps/OpenSignServer/cloud/parsefunction/pdf/PDF.js`): si `IsEnableOTP` es `true`, exige `request.user` (sesión Parse válida) antes de continuar; si es `false`, el endpoint confía en el `reqUserId` recibido como parámetro para identificar al firmante en `contracts_Contactbook`, sin exigir sesión.
- **Rechazo de documento** (`apps/OpenSignServer/cloud/parsefunction/declinedocument.js`): misma lógica — con `IsEnableOTP` en `false` acepta `request.params.userId` como identidad del firmante; con `IsEnableOTP` en `true` exige `request.user`.

En ambos casos, aunque no se exija sesión Parse, sí se valida que el `userId` / `reqUserId` provisto corresponda efectivamente a un firmante o al creador del documento (`declinedocument.js` verifica `isCreator` / `isSigner` contra el array `Signers` antes de aceptar la acción). El certificado de finalización marca "Security level: Email, OTP Auth" únicamente cuando `IsEnableOTP` fue `true` para ese documento (ver más abajo).

Generación de la firma (trazo, imagen, tipo por defecto)

El panel de firma (usado tanto en la auto-firma — `SignyourselfPdf.jsx` — como en la firma de un destinatario — `PdfRequestFiles.jsx` / `PlaceholderSign.jsx`) ofrece varias pestañas, cada una en `apps/OpenSign/src/components/pdf/tab/`:

- **Draw.jsx** — trazo a mano alzada con `react-signature-canvas`. Al soltar el trazo (`onEnd`), convierte el canvas a `dataURL` (`toDataURL()`) y lo pasa a `handleSignatureChange`. También soporta "prefill": si el firmante ya tiene un trazo guardado (`prefillImg` en el store Redux `widget`), lo carga con `canvasRef.current.fromDataURL(base64)`.
- **UploadImage.jsx** — permite subir una imagen (`image/png`, `image/jpeg`) como firma o sello (`isStampOrImage`), con un `<input type="file" hidden>` disparado por click en el recuadro de la firma.
- **DefaultSignature.jsx** (dos variantes: `components/pdf/DefaultSignature.jsx` para el panel de selección/confirmación, `components/pdf/tab/DefaultSignature.jsx` para mostrar la imagen ya elegida) — permite reutilizar la firma/inicial ya configurada por el usuario en su perfil (`defaultSignImg` / `myInitial` del store Redux), con una alerta de confirmación antes de aplicarla automáticamente a todos los widgets de firma/inicial del documento.

La configuración de firma "de base" del usuario (la que usa `ManageSign.jsx`, ruta `/managesign`) se persiste vía las Cloud Functions `savesignature / managesign` (`apps/OpenSignServer/cloud/parsefunction/saveSignature.js`, `manageSign.js`) sobre la clase `contracts_Signature`, con un puntero `UserId` a `_User` y campos `ImageURL`, `Initials`, `Stamp`, `SignatureName`. Ambas funciones exigen que `userId === request.user?.id` (un usuario no puede escribir la firma de otro). La lectura usa `getdefaultsignature` (`getSignature.js`), con la misma validación de propiedad. `apps/OpenSignServer/cloud/parsefunction/SignatureAfterFind.js` resuelve URLs firmadas para `ImageURL / Initials / Stamp` al leer.

Qué tipos de firma están habilitados globalmente para la cuenta (dibujo, imagen, default) se controla con `updatesignaturetype` (`apps/OpenSignServer/cloud/parsefunction/updatesignaturetype.js`), que persiste un array `SignatureType` en `contracts_Users` y **exige que quede al menos un tipo habilitado distinto de "default"** (rechaza con `INVALID_QUERY` si el único tipo activo es `default`).

Embebido de la firma y finalización del documento

La Cloud Function central de la ceremonia es `signPdf` (`apps/OpenSignServer/cloud/parsefunction/pdf/PDF.js`, registrada como `signPdf` en `main.js`). Por cada acción de firma:

1. Aplica los gates de identidad (OTP) y de orden estricto (`SendinOrder / SendInOrderStrict`) descritos arriba.
2. Agrega una entrada al array `AuditTrail` del documento: `{ UserPtr, SignedUrl: '', Activity: 'Signed', ipAddress }` (la IP sale de `x-real-ip`).
3. Determina si el documento quedó `isCompleted` comparando cuántas entradas de audit trail son "relevantes para completar" (`COMPLETION_ACTIVITIES`, incluye `Signed` y `Approved`) contra la cantidad de placeholders de firma/aprobación (excluyendo `Role === 'prefill'` y `viewers`).
4. Si quedó completo, arma la razón de firma con la lista de firmantes y **firma criptográficamente el PDF** usando `@signpdf/signpdf` + `@signpdf/signer-p12` + `@signpdf/placeholder-pdf-lib`: un certificado PKCS#12 (`.pfx`) — el propio del tenant si configuró `TenantId.PfxFile` (con su `password`), o el `PFX_BASE64 / PASS_PHRASE` global de variables de entorno como fallback — se usa para producir una firma digital embebida real en el PDF (no una imagen superpuesta).
5. Calcula `generateDocumentHash(signedDocs)` — un hash SHA-256 del buffer final firmado — y lo persiste en el campo `DocumentHash` del documento, únicamente cuando `isCompleted` es verdadero.

Certificado de finalización / audit trail

`generatecertificate` (`apps/OpenSignServer/cloud/parsefunction/generateCertificatebydocId.js`) sólo actúa si el documento está `IsCompleted` y todavía no tiene `CertificateUrl` (evita regenerar el certificado más de una vez). El contenido del certificado se arma en `apps/OpenSignServer/cloud/parsefunction/pdf/GenerateCertificate.js` con `pdf-lib`, y **sí contiene datos verificables reales**, no sólo texto decorativo:

- Id y nombre del documento.
- **Hash SHA-256 del documento** (`DocumentHash`), si existe.
- Organización, fecha de creación y fecha de completado (formateadas según `timezone`/formato de fecha del `ExtUserPtr` dueño).
- Cantidad de firmantes y datos del "document originator" (nombre, email, IP de origen del documento).
- Un bloque por cada entrada relevante del `AuditTrail`, ordenado cronológicamente por `SignedOn`, con: nombre, email, "Security level: Email, OTP Auth" (sólo si `IsEnableOTP` era `true` para ese documento), fecha de visualización (`ViewedOn`), fecha de firma (`SignedOn`), IP y la imagen de la firma embebida (o un placeholder "n/a" si no hay firma gráfica, por ejemplo en aprobaciones).

El PDF del certificado, una vez generado, **también se firma digitalmente** con el mismo mecanismo PKCS#12 (`pdfLibAddPlaceholder` + `SignPdf().sign(...)`) antes de subirse y guardar su URL en `CertificateUrl`.

Adicionalmente existe `apps/OpenSign/src/pages/VerifyDocument.jsx`, una utilidad de **verificación posterior** que permite subir cualquier PDF firmado y valida, en el propio navegador, la firma digital PKCS#7 embebida (lee el diccionario de firma, el `ByteRange`, recalcula el hash del contenido y lo compara contra el message digest almacenado en la firma). Es una verificación genérica de firma digital de PDF (independiente del campo `DocumentHash`/certificado propio de FibexSign) — se confirmó su existencia y su lógica de comparación de hash por lectura de código, pero un análisis exhaustivo de la implementación ASN.1/PKCS#7 queda fuera del alcance de este documento.

Rechazo del documento (decline)

`declinedoc` (`apps/OpenSignServer/cloud/parsefunction/declinedocument.js`) permite a un firmante o al creador rechazar un documento no completado (`IsCompleted` `≠ true`, `IsArchive` `≠ true`):

- Autoriza sólo si el llamante es el creador (`CreatedBy`) o figura en `Signers` .
- Setea `IsDeclined: true`, `DeclineReason`, `DeclineBy` (puntero al usuario que rechazó).
- Si quien rechaza no es el creador, notifica por email al creador (`declineMailer.send`, `sendDeclineMail`) con el motivo y un link de vuelta al documento.
- Es idempotente: si el documento ya estaba `IsDeclined`, devuelve `'document already declined'` sin volver a notificar.

Diagrama — ceremonia de firma de un firmante invitado

```
sequenceDiagram
    actor S as Firmante invitado
    participant FE as Frontend
    participant CC as Cloud Code
    participant DB as contracts_Document

    S->>FE: abre link de firma
    FE->>CC: SendOTPMailV1 (si aplica)
    CC-->>S: email con OTP
    S->>FE: ingresa OTP
    FE->>CC: /functions/AuthLoginAsMail
    CC-->>FE: sessionToken real de Parse
    FE->>FE: Parse.User.become(sessionToken)
    FE->>S: elige firma (Draw / UploadImage / DefaultSignature)
    S->>FE: confirma firma
    FE->>CC: Cloud Function "signPdf" {docId, reqUserId, pdfFile, signature}
    CC->>DB: valida IsEnableOTP (request.user si aplica)
    CC->>DB: valida SendinOrder + SendInOrderStrict
    alt orden no respetado
        CC-->>FE: 403 OPERATION_FORBIDDEN
    else autorizado
        CC->>DB: agrega entrada a AuditTrail (Signed, ipAddress)
        CC->>CC: ¿todas las entradas de completion cubiertas?
        alt documento completo
            CC->>CC: firma PDF con PKCS#12 (@signpdf + p12)
            CC->>CC: generateDocumentHash (SHA-256)
            CC->>DB: guarda SignedUrl, DocumentHash
            CC-->>FE: documento completado
            FE->>CC: Cloud Function "generatecertificate"
            CC->>CC: GenerateCertificate (hash, timestamps, audit trail)
            CC->>CC: firma certificado con PKCS#12
            CC->>DB: guarda CertificateUrl
        else faltan firmantes
            CC-->>FE: documento en progreso
        end
    end
end
```

Email Builder — Editor visual de plantillas de email

Este documento cubre el editor visual drag & drop para personalizar plantillas de email de solicitud y completitud, accesible en `/emailbuilder`. Se verificó contra `apps/OpenSign/src/pages/EmailBuilder.tsx`, `apps/OpenSign/src/components/emailbuilder/`, `apps/OpenSign/src/components/emaileditor/`, `apps/OpenSign/src/components/preferences/MailTemplateEditor.jsx`, `apps/OpenSignServer/cloud/parsefunction/createBatchDocs.js`, `apps/OpenSignServer/cloud/parsefunction/pdf/PDF.js`, y las Cloud Functions relacionadas.

Arquitectura general

El EmailBuilder es un editor visual independiente — **no tiene integración backend directa para guardar plantillas**. El flujo es:

1. Usuario abre `/emailbuilder` (ruta en `apps/OpenSign/src/App.jsx`)
2. Edita visualmente la plantilla con drag & drop
3. Exporta el resultado como HTML (copiar al portapapeles o descargar archivo)
4. Pega el HTML en `Preferences > Email` (`apps/OpenSign/src/components/preferences/tabs/Email.jsx`)
5. Guarda la plantilla vía Cloud Function `updatetenant` (persiste en `contracts_Users` / `contracts_Tenant`)

Alternativamente, puede exportar como JSON (serialización `#code/<base64>` para compartir links) o usar una plantilla de ejemplo (`#sample/requestemail`, `#sample/completionemail`).

Interfaz de usuario

Raíz: `apps/OpenSign/src/components/emailbuilder/App/index.tsx`. Estructura en tres paneles:

Toolbar (MainTabsGroup, DownloadJson, ShareButton, pantalla, toggles)		
SamplesDrawer (bloques disponibles)	TemplatePanel (editor visual)	Inspector Drawer (propiedades)
• Heading • Text • Button • Image • Divider • Columns • Spacer • HTML (custom)	[drag & drop aquí] [vista previa] [HTML/JSON code]	Tabs: - Block - Config - Styles

Componentes principales

- **SamplesDrawer** (`App/SamplesDrawer/`): panel izquierdo, lista de bloques (`heading` , `text` , `button` , `image` , `divider` , `columns` , `spacer` , `html`); drag & drop hacia `TemplatePanel`. Ancho: `SAMPLES_DRAWER_WIDTH = 280px` .
- **TemplatePanel** (`App/TemplatePanel/index.tsx`): área central; cuatro tabs:
 - `"editor"` — vista visual con drag & drop, canvas editable (componente `EditorBlock`)
 - `"preview"` — renderizado del email en un `iframe` (HTML sin interactividad)
 - `"html"` — vista de código HTML renderizado con `syntax highlighting`
 - `"json"` — vista de código JSON del documento (estructura de bloques)
- **InspectorDrawer** (`App/InspectorDrawer/`): panel derecho, dos tabs:
 - `"block-configuration"` — propiedades del bloque seleccionado (`TextInput`, `ColorInput`, `SliderInput`, etc., en `input-panels/helpers/inputs/`)
 - `"styles"` — estilos globales del email (ancho, fondo, tipografía, espaciado)
- **Botones/acciones:**
 - `ShareButton` — copia el HTML renderizado al portapapeles (notificación "Your current template is copied")
 - `DownloadJson` — descarga el HTML como archivo `emailTemplate.txt`
 - `Toggle de pantalla (Desktop/Mobile)` — cambia el viewport de edición

Gestión de estado

Estado en Zustand (`EditorContext.tsx`):

- `document` — la configuración del email (bloques, propiedades)
- `selectedBlockId` — id del bloque seleccionado actualmente
- `selectedMainTab` — pestaña activa (`editor/preview/html/json`)
- `selectedScreenSize` — viewport (`desktop/mobile`)
- `selectedSidebarTab` — tab del inspector (`block-configuration/styles`)
- `inspectorDrawerOpen` , `samplesDrawerOpen` — visibilidad de paneles

Carga de plantillas

Basada en `window.location.hash` (procesado en `getConfiguration/index.tsx`):

Patrón	Ejemplo	Comportamiento
<code>#sample/requestemail</code>	<code>/emailbuilder#sample/requestemail</code>	Carga plantilla de ejemplo "Solicitar firma" (<code>getRequestEmail()</code>)
<code>#sample/completionemail</code>	<code>/emailbuilder#sample/completionemail</code>	Carga plantilla de ejemplo "Documento completado" (<code>getCompletionEmail()</code>)
<code>#code/...</code>	<code>/emailbuilder#code/eyJ...</code>	Decodifica base64 → JSON (deserialization de plantillas guardadas via <code>EmailBodyEditor</code>)
(ninguno)	<code>/emailbuilder</code> o <code>/emailbuilder#</code>	Carga plantilla vacía (<code>EMPTY_EMAIL_MESSAGE</code>)

El frontend de `EmailBodyEditor` (`apps/OpenSign/src/components/EmailBodyEditor.jsx`) genera enlaces con `#sample/...` para permitir editar una plantilla de ejemplo:

```
href={`/${emailbuilder}${template}`} // template = "#sample/requestemail" | "#sample/completionemail" | "#"
```


Exportación y persistencia

Exportación a HTML

- **ShareButton**: genera HTML renderizado via `renderEmailHtml(document, "root")` → copia al portapapeles.
- **DownloadJson**: mismo HTML renderizado → descarga como `emailTemplate.txt`.
- Ambos usan `apps/OpenSign/src/components/emailbuilder/App/TemplatePanel/helper/renderEmailHtml.ts`, que transforma la estructura de bloques Zustand en HTML válido para email (compatible con clientes de email, inline CSS vía `juice`).

NO hay guardar directo desde el EmailBuilder. El usuario debe:

1. Copiar/descargar el HTML
2. Navegar a `Preferences > Email` (`apps/OpenSign/src/components/preferences/tabs/Email.jsx`)
3. Pegar en EmailBodyEditor (que ofrece dos modos: "basic" con Quill, "advanced" con textarea HTML)
4. Guardar via Cloud Function `updatetenant`

Exportación como JSON

- Tab "json" en TemplatePanel muestra `JSON.stringify(document, null, " ")`
- Seleccionable/copiable (HighlightedCodePanel con onclick handler)
- Puede serializarse manualmente a hash `#code/<base64(JSON)>` para compartir links (aunque no hay botón de "copy link" en la UI actual)

Persistencia en base de datos

Las plantillas se guardan **en las preferencias del usuario/tenant**, no en el EmailBuilder:

Cloud Function: `updatetenant` (backend)

Ubicación: `apps/OpenSignServer/cloud/parsefunction/updateTenant.js` (registrada como `updatetenant`)

Llamada desde: `MailTemplateEditor.jsx` en `handleSaveRequestEmail` / `handleSaveCompletionEmail`

Parámetros:

```
Parse.Cloud.run('updatetenant', {
  tenantId: tenantId,
  details: {
    RequestBody: '<html>...</html>', // HTML para email de solicitud
    RequestSubject: 'Firma requerida',
    CompletionBody: '<html>...</html>', // HTML para email de completitud
    CompletionSubject: 'Firmado',
    EmailEditorType: {
      request: 'basic' | 'advanced',
      completion: 'basic' | 'advanced'
    }
  }
})
```

Persistencia: campos en `contracts_Users` / `contracts_Tenant`:

- `RequestBody` — HTML de la plantilla de solicitud
- `RequestSubject` — asunto de solicitud
- `CompletionBody` — HTML de la plantilla de completitud
- `CompletionSubject` — asunto de completitud
- `EmailEditorType` — estado de qué editor se usó (básico/avanzado)

Nota de seguridad (ya documentada en `docs/cloud-functions-catalog.md`): la Cloud Function confía en el `tenantId` del cliente sin validación server-side adicional; valida que el usuario esté autenticado pero no verifica si tiene permisos en ese tenant específico.

Uso de las plantillas en el envío de emails

Las plantillas persistidas se recuperan y usan en el backend:

Email de solicitud

Ubicación: `apps/OpenSignServer/cloud/parsefunction/createBatchDocs.js` (envía emails cuando un documento se prepara para firma)

Lógica:

```
let mailBody = senderObj?.TenantId?.RequestBody || ''; // fallback a vacío
// ...reemplaza variables como {{signing_url}}, {{receiver_name}}, etc.
// ...envía via sendmailv3 con el HTML procesado
```

Email de completitud

Ubicación: `apps/OpenSignServer/cloud/parsefunction/pdf/PDF.js` (función `sendMailSaveCertificate`, se ejecuta tras firma final)

Lógica:

```
body = tenant?.CompletionBody ? tenant?.CompletionBody : body; // CompletionBody si existe, else default
// ...procesa, incrusta firma, genera certificado
// ...envía via sendMailWithAttachment
```

Ambas funciones usan `sendmailv3` (Cloud Function dedicada, ubicada en `apps/OpenSignServer/cloud/parsefunction/sendMailv3.js`) que:

- Soporta SMTP (via Nodemailer) o Mailgun (via SDK)
- Inyecta variables de reemplazo: `{{sender_name}}`, `{{receiver_name}}`, `{{document_title}}`, `{{signing_url}}`
- Incrusta el HTML en el cuerpo del mensaje y lo envía

Variables de reemplazo soportadas:

- `{{sender_name}}` — nombre del usuario que envía
- `{{receiver_name}}` — nombre del firmante
- `{{document_title}}` — título del documento
- `{{signing_url}}` — link de firma generado
- (otros según contexto del documento)

El frontend `MailTemplateEditor.jsx` no expone un editor visual para estas variables — se insertan manualmente en el HTML (o via placeholders en el `EmailBuilder` que luego se editan como texto).

Flujo completo

```
sequenceDiagram
    actor U as Usuario
    participant FE as Frontend
    participant EB as EmailBuilder (/emailbuilder)
    participant Prefs as Preferences > Email
    participant BE as Backend
    participant DB as contracts_Users
    participant Mailer as sendmailv3

    U->>EB: abre /emailbuilder (o #sample/requestemail)
    EB->>EB: carga plantilla ejemplo / vacía
    U->>EB: arrastra bloques, edita propiedades
    U->>EB: vista previa (desktop/mobile)
    U->>EB: copia HTML (ShareButton) o descarga (DownloadJson)
    U->>Prefs: navega a Preferences > Email
    U->>Prefs: pega HTML en EmailBodyEditor
    U->>Prefs: hace click "Guardar plantilla de solicitud"
    Prefs->>BE: Cloud.run('updatetenant', {RequestBody: ...})
    BE->>DB: guarda RequestBody en contracts_Users.RequestBody
    BE-->>Prefs: respuesta OK
    Prefs-->>U: "Guardado exitosamente"

    Note over U,U: Luego, al enviar un documento...
    U->>FE: crea documento y selecciona firmantes
    FE->>BE: crea documento (createBatchDocs)
    BE->>DB: lee RequestBody desde contracts_Users
    BE->>BE: reemplaza {{variables}}
    BE->>Mailer: sendmailv3({html: RequestBody, ...})
    Mailer-->>U: email recibido por firmante
```

Dos editores de email distintos

Existe potencial confusión entre dos componentes:

Aspecto	EmailBuilder	EmailEditor (Quill)
Ubicación	/emailbuilder ruta dedicada	Preferences > Email , tab "basic"
Tipo	Editor visual drag & drop (bloques: heading, text, button, image, etc.)	Editor de texto enriquecido (negritas, cursivas, listas, colores, fuentes)
Basado en	@usewaypoint/email-builder (componentes visuales, Zustand state)	Quill (rich text editor, Delta format)
Salida	HTML renderizado	HTML desde Quill Delta
Persistencia	Manual (copiar/pegar a Preferences)	Directo via updatetenant
Uso	Diseño complejo/profesional	Ediciones rápidas de texto

El usuario elige:

- **EmailBuilder** si quiere un layout profesional con múltiples columnas, imágenes, espaciado exacto.
- **EmailEditor (Quill)** si solo quiere editar texto rápidamente.

Ambos convergen en que el HTML final se persiste en `RequestBody` / `CompletionBody` y se envía igual por mail.

Notas técnicas

Renderización de HTML

`apps/OpenSign/src/components/emailbuilder/App/TemplatePanel/helper/renderEmailHtml.ts` transforma la estructura de bloques Zustand en HTML. Usa lógica custom (no delegado a `@usewaypoint/email-builder`, aunque ese paquete sí lo proporciona via `renderToStaticMarkup`):

- Procesa cada bloque recursivamente
- Genera `<div>`, `<p>`, `<img>`, `<a>`, `<table>` (para columnas) etc.
- Incrusta estilos inline (color, tamaño, fuente, padding)
- Genera un documento HTML válido (`<html><head>...</head><body>...</body></html>`)

Tema Material-UI

`apps/OpenSign/src/components/emailbuilder/EmailBuildertheme.ts` define un tema MUI para el editor (colores, tipografía, breakpoints), no relacionado con el tema del email final.

Requisitos de navegador

El EmailBuilder usa MUI, React hooks, Zustand — requiere navegador moderno con soporte ES2020+.

OpenSignDrive — Gestor de archivos y documentos

Este documento describe la arquitectura del OpenSignDrive, el espacio central de almacenamiento y organización de documentos en FibexSign. Cubre navegación de carpetas, operaciones de archivo (crear/renombrar/mover/eliminar), búsqueda, estados de documentos y la interacción con el backend. Verificado contra `apps/OpenSign/src/pages/Opensigndrive.jsx`, `apps/OpenSign/src/components/opensigndrive/DriveBody.jsx`, `apps/OpenSign/src/components/shared/fields/FolderModal.jsx`, y las Cloud Functions llamadas en `apps/OpenSignServer/cloud/parsefunction/`.

Navegación y estructura jerárquica

El Drive mantiene un árbol jerárquico de carpetas almacenadas como registros `contracts_Document` con `Type: 'Folder'` en Parse. La navegación ocurre en dos niveles:

1. **Breadcrumb de ruta** (`folderName` state en `Opensigndrive.jsx`): lista visual de navegación desde "OpenSignDrive" (raíz) hacia carpetas anidadas. Al hacer clic en un punto de la ruta, se recalcula el `docId` a ese nivel y se recargan los contenidos.
2. **Árbol de carpetas** (`FolderModal.jsx`): cuando se mueve un documento, se abre un modal que carga recursivamente subcarpetas mediante queries `Parse.Query` con `Folder` pointer (filtro: `Type: 'Folder'`, `IsArchive !== true`, `CreatedBy: currentUser`).

La relación padre-hijo se modela con un puntero `Folder` en `contracts_Document`:

```
// Carpeta raíz: no tiene campo Folder (doesNotExist("Folder"))
// Carpeta anidada: Folder -> { __type: "Pointer", className: "contracts_Document", objectId: parentId }
```

Listado de documentos y carpetas

`Opensigndrive.jsx` invoca `getDrive(docId, skip, limit)` (utility wrapper en `apps/OpenSign/src/constant/Utils.js` → Cloud Function `getDrive`) para cargar:

- **Parámetros:** `docId` (carpeta actual, opcional; si no se pasa, lista documentos en raíz), `skip` (paginación), `limit` (50 por defecto)
- **Retorno:** array de `contracts_Document` (tanto carpetas como documentos), excluye campos sensibles (`AuditTrail`, `OriginalDocument`, `SignedDocument`)
- **Filtrado:** server-side por `CreatedBy = request.user` + `IsArchive !== true`

La paginación usa **infinite scroll** con `IntersectionObserver` (`ref bottomRef`) — cuando el usuario llega al final, automáticamente se carga la siguiente página sin recargar la UI.

Los documentos se renderizan en dos modos de vista (controlados por `isList` state):

1. **Cuadrícula** (`!isList`): cards con vista previa de estado, icono PDF/carpetas, nombre truncado, fecha, y hover card con detalles (signatarios, estado exacto).
2. **Tabla** (`isList`): filas con columnas Nombre, Fecha creación, Tipo, Estado, Acción (descargar).

Operaciones sobre carpetas y documentos

Crear carpeta

`handleAddFolder` (wrapped con `withSessionValidation`, línea 277):

1. Valida que el nombre no sea vacío.
2. Consulta si ya existe una carpeta con ese nombre en el mismo nivel (`Parse.Query` con `Name`, `Type: 'Folder'`, padre `Folder` opcional, `IsArchive !== true`).
3. Si no existe, crea un nuevo `Parse.Object` con:
 - `Type: 'Folder'`
 - `Name: newFolderName`
 - `ExtUserPtr` → puntero al `contracts_Users` del usuario actual
 - `CreatedBy` → puntero al `_User` del usuario actual
 - `Folder` (opcional) → puntero al padre si está anidada
4. Persiste localmente en `pdfData` y re-ordena según los parámetros de sorting actuales.

Renombrar documento/carpeta

`handledRenameDoc` (línea 55):

1. Valida longitud > 0.
2. Actualiza localmente el array `pdfData` con el nuevo nombre.
3. Envía un `PUT` REST directo a `/classes/contracts_Document/:docId` con `{ Name: newFolderName }` y headers de sesión (`X-Parse-Session-Token`).

Nota: esto es una actualización REST directa, no una Cloud Function, lo que implica que confía en la ACL/sesión de Parse para autorización.

Mover documento/carpeta

`handleMoveFolder` (línea 195):

1. Abre un modal (`FolderModal.jsx`) que permite navegar la jerarquía de carpetas.
2. El usuario selecciona una carpeta destino (o raíz).
3. Detecta si el documento ya está en esa carpeta (compara `moveFolderId` con `selecFolderId`) para evitar no-ops.
4. Envía `PUT /classes/contracts_Document/:docId` con:
 - Si hay destino: `{ Folder: { __type: "Pointer", className: "contracts_Document", objectId: moveFolderId } }`
 - Si es raíz: `{ Folder: { __op: "Delete" } }` (elimina el puntero)
5. Remueve el documento del array `pdfData` en el frontend.

Eliminar carpeta

`handleDeleteFolder` (línea 275):

1. Verifica que la carpeta esté vacía mediante `checkFolderEmpty` (query Parse por `Folder` pointer, excluye archivados).
2. Si está vacía, obtiene el objeto y establece `IsArchive: true` (soft delete vía Parse SDK).
3. Si no está vacía, muestra una notificación de error.

Eliminar documento

`handleDeleteDocument` (línea 158):

1. Envía `PUT /classes/contracts_Document/:docId` con `{ IsArchive: true }`.
2. Remueve del array `pdfData` localmente.

Nota: no hay borrado físico — todos los documentos/carpetas se marcan con `IsArchive: true` para auditoría.

Estados de documento

`getStatusBadgeConfig` (línea 301) determina el estado según los campos del documento:

Campo	Condición	Estado
<code>IsCompleted</code>	<code>true</code>	"Completed" (verde)
<code>IsDeclined</code>	<code>true</code>	"Declined" (rojo)

SignedUrl ausente	true	"Draft" (azul)
Fecha expiración < ahora	true	"Expired" (gris)
(otro)	—	"In Progress" (ámbar)

El estado se usa para renderizar un badge con icono y color, y para determinar a qué página navegar cuando se hace clic sobre un documento (ver `checkPdfStatus` , línea 95).

Búsqueda de documentos

`handleSearchChange` (línea 530) invoca `Parse.Cloud.run('filterdocs', { searchTerm })` con un debounce de 300ms:

- **Parámetro:** `searchTerm` (búsqueda case-insensitive sobre `Name`)
- **Retorno:** array de `contracts_Document` que coinciden (desde el backend, filtrados por `CreatedBy = request.user`)
- **Comportamiento:** reemplaza completamente `pdfData` con los resultados; al borrar el término, llama nuevamente con string vacío que devuelve todos los documentos del usuario.

Integración con flujos de firma

El Drive **no inicia** la preparación ni el envío de documentos. Solo navega documentos existentes. Cuando se hace clic sobre un documento, `checkPdfStatus` determina a qué pantalla redirigir:

```
// Draft (sin destinatarios ni placeholders): /signaturePdf/:docId (auto-firma)
// In progress (con placeholders + SignedUrl): /recipientSignPdf/:docId (ver estado de firma)
// Draft con destinatarios/placeholders: /placeholderSign/:docId (editar/enviar)
// IsSignyourself: /signaturePdf/:docId
// Etc.
```

Para crear documentos nuevos, el usuario usa el botón "+ Nuevo" que abre un menú con:

- **Crear carpeta** → abre modal de entrada de nombre
- **Sign Yourself** → redirige a `/form/sHAnZphf69` (plantilla de auto-firma)
- **Request Signatures** → redirige a `/form/8mZzFxbG1z` (plantilla de envío a terceros)

Estas son plantillas predefinidas, no flujos de creación desde cero.

Ordenamiento

`sortingData` (línea 365) y `sortedBy` (línea 344) permiten ordenar por:

- **Nombre:** ascendente/descendente (case-insensitive)
- **Fecha creación:** ascendente/descendente

El estado se mantiene en `selectedSort` y `sortingOrder` , y aplica a toda la lista renderizada cada vez que cambia la selección.

Estados y pagination

- `pdfData` : array completo de documentos cargados hasta ahora (acumula conforme se hace scroll).
- `skip` : offset de paginación (incrementa en `limit` pasos, default 50).
- `loading` : flag de carga en vuelo para evitar requests duplicadas.
- `folderName` : breadcrumb actual.
- `docId` : ID de la carpeta actual siendo visualizada (undefined = raíz).

Al navegar a una carpeta diferente, se resetean `skip: 0` y `pdfData: []` para cargar desde cero.

Tour interactivo (primera visita)

Al primer acceso, `Opensigndrive.jsx` llama `checkTourStatus` que invoca `Parse.Cloud.run('getUserDetails')` para revisar si el usuario ya vio el tour (campo `TourStatus`). Si no, renderiza un tour paso a paso con `reactour` (7 pasos configurables). El usuario puede marcar "Don't show again" (`isDontShow`), que persiste vía `PUT /classes/contracts_Users/:extUserId` con `{ TourStatus: [...] }`.

Diagrama — flujo de navegación y operaciones en OpenSignDrive

```
flowchart TD
    A[Usuario accede a /opensigndrive] --> B[Opensigndrive.jsx]
    B --> C[¿Primer acceso?]
    C -->|sí| D[getUserDetails → verificar TourStatus]
    C -->|no| E[getDrive → cargar documentos de raíz]
    D --> E
    E --> F[Renderizar carpetas + documentos<br/>en grid o tabla]
    F --> G[Acción del usuario]
    G -->|Click en carpeta| H[Actualizar docId + folderName<br/>getDrive con nuevo docId]
    G -->|Click en documento| I[checkPdfStatus →<br/>navegar a /signaturePdf, /recipientSignPdf, etc.]
    G -->|Búsqueda| J[filterdocs → resultados en pdfData]
    G -->|+ Nuevo| K[Menú: Crear carpeta / Sign Yourself / Request Signatures]
    K -->|Crear carpeta| L[Validar nombre único a este nivel<br/>Parse.Object.save con Type: Folder]
    K -->|Sign Yourself| M[Redirigir a /form/sHAnZphf69]
    K -->|Request Signatures| N[Redirigir a /form/8mZzFxbG1z]
    L --> O[Renombrar| O[REST PUT /classes/contracts_Document<br/>{ Name: newName }]]
    O --> P[FolderModal.jsx → seleccionar destino<br/>REST PUT con Folder pointer o __op: Delete]
    P --> Q[Eliminar| Q[¿Es carpeta?]]
    Q -->|sí| R[Verificar isEmpty<br/>REST PUT { IsArchive: true }]
    Q -->|no| S[REST PUT { IsArchive: true }]
    R --> F
    S --> F
    J --> F
    I --> F
    H --> F
    K --> F
    L --> F
    M --> F
    N --> F
    O --> F
    P --> F
    Q --> F
    R --> F
    S --> F
```

Cloud Functions utilizadas

Verificadas en `docs/cloud-functions-catalog.md`:

- `getDrive` (línea 121 del catálogo): lista documentos/carpetas de una carpeta con paginación.
- `filterdocs` (línea 163): busca documentos por nombre.
- `getUserDetails` (línea 28): obtiene detalles extendidos del usuario (incluyendo `TourStatus`).

Ninguna de estas funciones se documentaba previamente específicamente para OpenSignDrive, pero todas están presentes en el catálogo.

Operaciones Parse SDK (no Cloud Functions)

El Drive también usa consultas Parse SDK directas sobre `contracts_Document`:

Operación	Línea aprox.	Propósito
<code>Parse.Query("contracts_Document").equalTo("Name", ...)</code>	296–302 (Opensigndrive)	Verificar nombre de carpeta único antes de crear
<code>Parse.Query("contracts_Document").equalTo("Folder", folderPtr)</code>	260 (DriveBody)	Listar contenido de una subcarpeta
<code>Parse.Query("contracts_Document").doesNotExist("Folder")</code>	36 (FolderModal)	Listar documentos en raíz del drive

Autenticación y sesión

Todas las operaciones que escriben (`handleAddFolder` , `handleRenameDoc` , `handleMoveFolder` , `handleDeleteDocument` , `handleDeleteFolder`) están wrappedadas con `withSessionValidation` , que verifica:

- `localStorage.TenantId` existe
- `Parse.User.current().getSessionToken()` es válido

Si falta alguno, aborta la operación y despacha `sessionStatus(false)` a Redux, que dispara `SessionExpiredModal` y cierra sesión.

Hallazgos y discrepancias

1. **Operaciones REST directas sin Cloud Function:** Renombrar, mover y eliminar usan `axios.put` contra `/classes/contracts_Document/:docId` en vez de Cloud Functions dedicadas. Esto significa que la autorización se confía completamente a la ACL de Parse y al `X-Parse-Session-Token` enviado en headers. Esto es funcional pero menos auditable que si hubiese funciones centralizadas (`renameDocFromDrive` , `moveDocFromDrive` , etc.) que logueen cambios específicamente.
2. **Soft delete sin cascada auditable:** `IsArchive: true` es el único mecanismo de eliminación. Dependiendo de qué tan estrictos sean los requisitos de auditoría, puede ser necesario un hook `beforeSave` que registre en un log de cambios (audit trail) cada vez que se archiva, especialmente para carpetas que contienen documentos sensibles.
3. **No hay validación server-side de nombre único:** La validación de "carpeta ya existe" ocurre client-side (`Parse.Query` en el cliente) antes de crear. Aunque Parse resuelve la sesión, es posible que dos requests concurrentes pasen la check simultáneamente y creen duplicados si no hay un índice único DB-side sobre (`Folder` , `Name` , `CreatedBy` , `IsArchive`) .

Reportes — contactos, documentos y plantillas

Este documento describe los tres tipos de reportes disponibles en FibexSign — documentos, contactos y plantillas — las vistas que los muestran, los filtros y búsquedas disponibles, y cómo se genera y exporta el contenido. Todo fue verificado contra `apps/OpenSign/src/pages/Report.jsx`, `apps/OpenSign/src/json/ReportJson.js`, `apps/OpenSign/src/reports/{contact,document,template}/`, `apps/OpenSignServer/cloud/parsefunction/getReport.js` y `reportsJson.js`.

Punto de entrada y enrutamiento

El usuario llega a los reportes desde `/report/:id`, donde `:id` es un identificador de reporte registrado en `ReportJson.js` (frontend) y `reportsJson.js` (backend). El componente `Report.jsx` actúa como orquestador: resuelve qué vista específica renderizar según el `id`, carga la configuración de columnas desde `localStorage` (clave: `reportColumns`), y delega las llamadas de datos a la Cloud Function `getReport`.

```
GET /report/:id
├─ report id = "contacts" → Contactbook.jsx
├─ report id = "6TeaPr321t" → TemplatesReport.jsx
└─ report id = (documento) → DocumentsReport.jsx
```

Reportes de documentos

Tipos de reportes documentales

Frontend (`ReportJson.js`) y backend (`reportsJson.js`) definen 9 tipos de reportes sobre `contracts_Document`:

ID	Nombre	Descripción
ByHuevtCFY	Draft Documents	Documentos sin enviar
4Hhwbp482K	Need your sign	Documentos esperando mi firma
1MwEuxLEkF	In-progress documents	Documentos enviados, en progreso
kQUoW4hUXz	Completed Documents	Documentos completamente firmados
UPr2Fm5WY3	Declined Documents	Documentos rechazados
zNqBHXHsYH	Expired Documents	Documentos vencidos
d9k3UfYHBc	Recently sent for signatures	Últimos enviados (dashboard)
5Go51Q7T8r	Recent signature requests	Últimos que me piden firmar (dashboard)
kC5mfynCi4	Drafts	Borradores (dashboard)

Columnas mostradas

Cada reporte muestra un conjunto fijo de columnas (configurables por usuario en `localStorage`):

- **Comunes:** `Title`, `Note`, `Folder`, `File`, `Owner`, `Signers`
- **In-progress:** `Title`, `Note`, `Folder`, `File`, `Signers`, `Sent Date`
- **Declined:** `Title`, `Reason`, `Folder`, `File`, `Owner`, `Signers`
- **Extra columnas** (seleccionables): `Note`, `Time to complete (Days)`, `Enable Tour`, `Notify on signatures`, `Redirect url`, `Created Date`, `Updated Date`

Acciones por reporte

- **Draft Documents:** Edit, Delete
- **Need your sign:** Sign
- **In-progress:** Share, View, Resend, Rename, Extend expiry, Revoke, Save as template, Delete
- **Completed Documents:** View, Delete, Save as template, Fix & resend
- **Declined Documents:** View, Delete, Save as template, Fix & resend
- **Expired Documents:** View, Delete, Extend expiry, Save as template

Búsqueda y filtros

- **Búsqueda:** campo libre sobre `Name` (debounce 300ms, AbortController)
- **Filtro de estado de firmante:** `signerStatus` (all/viewed/signed)
- **Paginación:** `skip` y `limit`, típicamente 20 documentos por llamada

Reporte de contactos (Contactbook)

Vista: `apps/OpenSign/src/reports/contact/Contactbook.jsx`

Datos mostrados

Columnas: `Name`, `Email`, `Phone`, `Company`, `JobTitle`

Origen: `contracts_Contactbook` del usuario autenticado, filtrados por `IsDeleted !== true`.

Acciones

- **Edit:** abre `EditContactForm.jsx`, llama Cloud Function `editcontact`
- **Delete:** soft-delete con `IsDeleted: true`

Importación de contactos (ImportContact.jsx)

- **Formatos:** `.csv`, `.xlsx`, `.xls` (máx. 100 registros)
- **Headers requeridos:** `Name`, `Email`
- **Headers opcionales:** `Phone`, `Company`, `JobTitle`
- **Validación:** regex de email
- **Envío:** Cloud Function `createbatchcontact`

Reporte de plantillas (TemplatesReport)

Vista: `apps/OpenSign/src/reports/template/TemplatesReport.jsx`

Datos mostrados

Columnas: `Title`, `File`, `Owner`, `Signers` (base)

Origen: `contracts_Template` filtrado por `Type !== 'Folder'` y `IsArchive !== true`

Acciones

- **Use:** crea documento nuevo desde plantilla
- **Quick send:** envío masivo
- **Opciones:** Edit, Rename, Delete, Share with team

Tour interactivo

`ReportTour.jsx` define 4 pasos de onboarding (selector data-tut):

1. `reactourFirst` - Introducción
2. `reactourSecond` - Botón "Use"
3. `tourbulksend` - Botón "Quick send"
4. `reactourThird` - Menú opciones

Cloud Function `getReport`

Archivo: `apps/OpenSignServer/cloud/parsefunction/getReport.js`

Parámetros:

- `reportId` (required): identificador del reporte
- `skip` (required): offset de paginación
- `limit` (required): cantidad de resultados
- `searchTerm` (optional): búsqueda por Name/Email
- `signerStatus` (optional): viewed/signed/all
- Headers: `sessiontoken` o `x-parse-session-token`

Flujo interno:

1. Resuelve usuario vía `GET /users/me`
2. Llama `reportJson(reportId, userId)` para obtener params y clase
3. Aplica filtros especiales para Templates (equipos, usuarios compartidos)
4. Aplica `applySearch()` si hay `searchTerm`
5. Filtra por `AuditTrail.Activity` si `signerStatus` está presente
6. Construye URL REST a `/classes/{reportClass}` y retorna resultados

Filtro de búsqueda (`applySearch`)

Definido en `reportsJson.js`:

- **Contactbook:** `$or: [{ Name: /searchTerm/i }, { Email: /searchTerm/i }]`
- **Documentos/Plantillas:** `$or: [{ Name: /searchTerm/i }, { Signers: { $inQuery: { ... Email: /searchTerm/i } } }]`

Selección de columnas

Almacenamiento: `localStorage.reportColumns` = JSON con estructura:

```
{
  "reportId": {
    visible: ["Name", "Email"],
    labels: { "Name": "Full Name" }
  }
}
```

Fallback: si no hay preferencia, se usan columnas por defecto de `json.heading`

Integración con backend (`reportsJson.js`)

Archivo: `apps/OpenSignServer/cloud/parsefunction/reportsJson.js`

Cada `reportId` retorna:

```
{
  reportName: "...",
  reportClass: "contracts_Document" | "contracts_Contactbook" | "contracts_Template",
  params: { /* query filters */ },
  keys: [ /* columnas a retornar */ ]
}
```

- `params` : filtros WHERE de Parse
- `keys` : proyección de columnas
- Acceso acotado a usuario autenticado
- Templates: acceso por equipos + usuarios compartidos

Internacionalización (i18n)

FibexSign soporta **7 idiomas** mediante `i18next` + `react-i18next` con carga dinámica de archivos de traducción vía HTTP. Este documento explica cómo se configura y cómo agregar nuevos idiomas o traducciones.

Configuración verificada contra:

- `apps/OpenSign/src/i18n.js`
- `apps/OpenSign/package.json`
- `apps/OpenSign/public/locales/{de,en,es,fr,hi,it,kr}/translation.json`
- `apps/OpenSign/src/components/pdf/SelectLanguage.jsx`

Configuración de i18next

El archivo `apps/OpenSign/src/i18n.js` inicializa i18next con la siguiente topología:

```
import i18n from "i18next";
import { initReactI18next } from "react-i18next";
import Backend from "i18next-http-backend";
import LanguageDetector from "i18next-browser-languagedetector";

i18n
  .use(Backend)
  .use(LanguageDetector)
  .use(initReactI18next)
  .init({
    backend: {
      loadPath: "/locales/{lng}/{ns}.json"
    },
    fallbackLng: "en",
    detection: {
      order: ["localStorage", "navigator"],
      caches: ["localStorage"]
    },
    ns: ["translation"],
    defaultNS: "translation",
    debug: false,
    interpolation: {
      escapeValue: false
    },
    whitelist: ["en", "es", "fr", "it", "de", "hi", "kr"]
  });
```

Módulos utilizados

Paquete	Versión	Propósito
i18next	^26.0.8	Motor central de internacionalización
react-i18next	^17.0.6	Integración con React (hooks, componentes)
i18next-http-backend	^3.0.6	Carga de archivos de traducción vía HTTP (no bundleados)
i18next-browser-languagedetector	^8.2.1	Detección automática de idioma del navegador y persistencia en <code>localStorage</code>

Comportamiento clave

- **Carga de archivos:** Las traducciones se cargan dinámicamente desde `/locales/{{lng}}/{{ns}}.json` (ej. `/locales/es/translation.json`). No están incluidas en el bundle — se descargan al navegador cuando cambia el idioma.
- **Namespace:** Solo existe un único namespace llamado `translation`. Todas las claves de traducción viven en un único archivo `translation.json` por idioma.
- **Detección de idioma:** El sistema intenta detectar el idioma del usuario en este orden:
 1. Valor guardado en `localStorage` bajo la clave `i18nextLng` (persiste entre sesiones).
 2. Idioma del navegador (vía `navigator.language` o `navigator.languages`).
 3. Fallback a `en` si ninguno de los anteriores está disponible o no es soportado.
- **Interpolación:** Las traducciones pueden incluir variables con sintaxis `{{variable}}` (ej. `"Subscription renews in {{days}} days"`). React i18next escapa automáticamente el HTML, así que no hay riesgo de XSS.

Estructura de archivos de localización

```
apps/OpenSign/public/locales/
├── de/
│   └── translation.json      # Alemán (Deutsch)
├── en/
│   └── translation.json      # Inglés (English)
├── es/
│   └── translation.json      # Español
├── fr/
│   └── translation.json      # Francés (Français)
├── hi/
│   └── translation.json      # Hindi (हिन्दी)
├── it/
│   └── translation.json      # Italiano
└── kr/
    └── translation.json      # Coreano (한국어)
```

Cada archivo `translation.json` contiene un objeto JSON plano con ~1300 pares clave-valor. Las claves son **kebab-case** (ej. `enter-document-title`, `user-already-exist`). Algunos valores pueden ser strings simples o objetos anidados para agrupar conceptos relacionados:

```
{
  "language": "Language",
  "email": "Email",
  "sort-order": {
    "Ascending": "Ascending",
    "Descending": "Descending",
    "Name": "Name",
    "Date": "Date"
  },
  "context-menu": {
    "Download": "Download",
    "Rename": "Rename",
    "Move": "Move",
    "Delete": "Delete"
  },
  "subscription-renew-warning": "Your subscription will expire in {{remainingDays}} days. Please renew your subscription."
}
```

Uso en componentes React

Hook `useTranslation`

Todos los componentes que necesiten acceso a traducciones importan y usan el hook:

```
import { useTranslation } from "react-i18next";

function MyComponent() {
  const { t, i18n } = useTranslation();

  return (
    <div>
      <h1>{t("document-details")}</h1>
      <button onClick={() => i18n.changeLanguage("es")}>
        Español
      </button>
    </div>
  );
}
```

Ejemplos en el codebase:

- `apps/OpenSign/src/components/AddUser.jsx:23` — `const { t } = useTranslation();`
- `apps/OpenSign/src/components/AddUser.jsx:71` — `notify.warning(t("valid-email-alert"));`
- `apps/OpenSign/src/components/AddUser.jsx:77` — `notify.error(t("user-already-exist"));`

Cambio de idioma

El componente `SelectLanguage` (ubicado en `apps/OpenSign/src/components/pdf/SelectLanguage.jsx`) proporciona la interfaz de usuario para cambiar idioma:

```
const handleChangeLang = (e) => {
  const newLang = e.target.value;
  i18n.changeLanguage(newLang);
  updateExtUser && updateExtUser({ language: newLang });
};
```

Cuando el usuario selecciona un idioma nuevo:

1. `i18n.changeLanguage(newLang)` dispara la descarga del archivo `translation.json` del nuevo idioma (si no está cacheado en memoria).
2. Todos los componentes que usan `useTranslation()` se re-renderizan automáticamente con las nuevas cadenas.
3. Se persiste el idioma elegido en `localStorage` para la próxima sesión.

Agregar un nuevo idioma

Para agregar un idioma nuevo (ej. `pt` para Portugués):

1. Crear la carpeta y archivo de traducción:

```
mkdir -p apps/OpenSign/public/locales/pt
cp apps/OpenSign/public/locales/en/translation.json apps/OpenSign/public/locales/pt/translation.json
```

2. **Traducir las claves:** Abre `apps/OpenSign/public/locales/pt/translation.json` y reemplaza los valores en inglés con las traducciones en portugués. Mantén las claves exactamente igual — solo traducen los valores.

3. **Actualizar la configuración de i18next:** Edita `apps/OpenSign/src/i18n.js` y agrega el código al array `whitelist`:

```
whitelist: ["en", "es", "fr", "it", "de", "hi", "kr", "pt"]
```

4. **Registrar el idioma en el selector UI:** Edita `apps/OpenSign/src/components/pdf/SelectLanguage.jsx` y agrega una entrada al array `LANGUAGES`:

```
const LANGUAGES = [
  { value: "en", text: "English" },
  { value: "es", text: "Español" },
  // ...
  { value: "pt", text: "Português" }
];
```

5. **Verificar:** Reinicia el servidor de desarrollo (`pnpm dev:frontend` en `apps/OpenSign`) y la nueva opción debe aparecer en el selector de idioma.

Nota sobre `whitelist` vs `supportedLngs`: El código actual usa `whitelist`, que es la opción de versiones antiguas de i18next. A partir de la versión 24+, se reemplazó por `supportedLngs`. En la versión instalada (`^26.0.8`), `whitelist` sigue funcionando pero genera un aviso de deprecación — para futuros refactoring, considerar migrar a `supportedLngs`.

Agregar una clave de traducción nueva

Cuando se agrega una característica nueva que requiere texto traducible:

1. **Decidir el nombre de la clave:** Usa kebab-case siguiendo la convención actual (ej. `new-feature-title`, `delete-confirmation-message`).
2. **Agregar la clave a TODOS los idiomas:** Abre cada archivo `translation.json` en los 7 locales y agrega la nueva clave:

```
{
  "existing-key": "...",
  "new-feature-title": "Your translated text here",
  ...
}
```

3. **Usar en el componente:**

```
const { t } = useTranslation();
return <h1>{t("new-feature-title")}</h1>;
```

Problema actual — sin automatización de sincronización: No hay scripts en `apps/OpenSign/package.json` que detecten automáticamente:

- Claves faltantes en algún idioma (ej. la clave `attachments-docx-size-limit` falta en `es` pero existe en `en`).
- Claves obsoletas que fueron eliminadas del código pero quedan en los archivos de traducción.
- Nuevas claves en el código que no han sido agregadas a todos los locales.

Hallazgo verificado: Comparando `translation.json` entre `en` y `es`, existe una discrepancia:

- `es` tiene `attachments-docx-upload-limit` pero falta `attachments-docx-size-limit`.
- Esto puede causar que esa clave muestre la clave misma (ej. `"attachments-docx-size-limit"`) en lugar de texto traducido cuando se usa en `es`.

Recomendación para el futuro: Considerar agregar un script de validación (ej. con `node` o `bash`) que:

- Lea todos los archivos de `translation.json`.
- Detecte claves faltantes entre idiomas.
- Alerta al desarrollador si una clave nueva aparece en el código pero no en todos los `translation.json`.

Cambio de idioma persistente

Cuando un usuario autenticado cambia su idioma de preferencia vía `SelectLanguage`, el componente llama a `updateExtUser({ language: newLang })`, que típicamente:

1. Persiste la preferencia en la base de datos del usuario (vía un Cloud Function o API endpoint del backend).
2. En sesiones futuras, `HomeLayout.jsx` o `Login.jsx` lee esa preferencia y llama a `i18n.changeLanguage()` automáticamente.

Esto asegura que cada usuario vea su idioma preferido sin necesidad de confiar únicamente en `localStorage`.

Cacheado del navegador

Una vez que se descarga un archivo `translation.json`, el navegador lo cachea en memoria por la duración de la sesión. El cambio entre idiomas es instantáneo porque:

1. Primera carga de idioma → descarga vía HTTP (Backend de `i18next-http-backend`).
2. Cambios posteriores del mismo idioma → lectura de memoria (sin HTTP).

Cambiar a un idioma nuevo que ya fue usado antes en esa sesión es de nuevo instantáneo.

Interpolación de variables

Las claves de traducción pueden incluir placeholders que se reemplazan en runtime:

```
{
  "subscription-renew-warning": "Your subscription will expire in {{remainingDays}} days. Please renew your subscription.",
  "gdrive-info-connect": "When Google Drive is connected, the completed document will be saved in the {{appName}} folder on C",
}
```

En el componente, se pasan las variables como segundo argumento a `t()`:

```
{t("subscription-renew-warning", { remainingDays: 7 })}
{t("gdrive-info-connect", { appName: "FibexSign" })}
```

React i18next escapa automáticamente los valores, previniendo inyecciones de XSS.

Locale codes

Los códigos de idioma usados son códigos ISO 639-1 de dos caracteres (con una excepción: `kr` para Coreano en lugar del estándar `ko`). Esta discrepancia es un legado del fork de OpenSign original y se mantiene por compatibilidad.

Código	Idioma	Nota
--------	--------	------

en	English	Idioma por defecto, usado como fallback
es	Español	—
fr	Français	—
it	Italiano	—
de	Deutsch	—
hi	Hindi	—
kr	한국어 (Coreano)	Debería ser ko según ISO 639-1, pero se usa kr

Arquitectura del Frontend

Este documento describe la estructura, capas y patrones de organización del código frontend de FibexSign. Todo lo aquí descrito fue verificado contra `apps/OpenSign/src/` (entry point `App.jsx`, enrutamiento, estado con Redux y Context, hooks custom, componentes por feature, estilos con Tailwind/DaisyUI/MUI/Radix, y primitivos UI).

Entry point y enrutamiento

`apps/OpenSign/src/App.jsx` es el punto de entrada: inicializa Parse SDK, carga el worker de `pdfjs-dist`, configura las variables locales de `baseUrl` y `parseAppId`, y define todas las rutas de la aplicación usando React Router v7 (`BrowserRouter` + `Routes` / `Route`).

Lazy loading con `lazyWithRetry`

Las páginas se cargan bajo demanda mediante `React.lazy()` envuelto en `lazyWithRetry` (`apps/OpenSign/src/Utils/lazyWithRetry.js`), una utilidad que captura errores de importación dinámica (típicamente causados por deploy nuevos mientras el cliente aún tiene la versión vieja en caché). Cuando detecta `/Failed to fetch dynamically imported module/`, guarda un flag en `localStorage.showUpgradeProgress`, recarga la página y muestra un progreso de actualización. Ejemplo en `App.jsx`:

```
const ForgetPassword = lazyWithRetry(() => import("./pages/ForgetPassword"));
const GuestLogin = lazyWithRetry(() => import("./pages/GuestLogin"));
```

Todas las rutas lazy son envueltas en el componente `Lazy` (`apps/OpenSign/src/primitives/LazyPage.jsx`), que renderiza una pantalla de carga (`Loader`) dentro de un `Suspense` mientras la página se descarga.

Rutas y guards

Las rutas se organizan en tres árboles principales:

1. Routes públicas sin sesión:

- `<Route element={<ValidateRoute />}>` — envuelve Login, AddAdmin, Upgrade. `ValidateRoute` (`apps/OpenSign/src/primitives/ValidateRoute.jsx`) comprueba si existe `localStorage.accesstoken` y valida el token contra Parse; si es inválido, ejecuta `handlelogout()` (borra `localStorage` pero preserva configuración de tema, idioma y URLs).
- `<Route path="/login/:base64url">` y `<Route path="/debugpdf">` — rutas sueltas, sin guardia (acceso público).

2. Rutas protegidas para firmantes invitados:

- `<Route element={<Validate />}>` — envuelve el flujo de firma de invitados (`/load/recipientSignPdf`). `Validate` (`apps/OpenSign/src/primitives/Validate.jsx`) verifica sesión y renderiza `<SessionExpiredModal />` si falta `localStorage.accesstoken` o es inválido, leyendo el usuario almacenado desde `localStorage.Parse/{appId}/currentUser`.

3. Rutas protegidas autenticadas:

- `<Route element={<HomeLayout />}>` — envuelve el dashboard, perfil, drive, preferencias y todas las rutas que requieren usuario logueado. `HomeLayout` (`apps/OpenSign/src/layout/HomeLayout.jsx`) renderiza header, sidebar y outlet; si la sesión caduca, el dispatch de `sessionStatus(false)` a Redux fuerza que `HomeLayout` muestre `SessionExpiredModal` en lugar del contenido.

State management — Redux + Context

Redux Toolkit (`apps/OpenSign/src/redux/`)

El store centralizado maneja seis dominios de estado:

- `user` (`userReducer.js`): `userInfo`, `isValidSession`, `isLoading`, `isTopLoader`, `tenantInfo`, `alertInfo`. Actions: `setUserInfo`, `sessionStatus`, `setTenantInfo`, `setLoader`, `setTopLoader`, `setAlertInfo`.
- `appInfo` (`infoReducer.js`): información global de la app (probablemente configuración de tenant, estado de upgrade).
- `TourSteps` (`TourStepsReducer.js`): control de onboarding/tutorial.
- `ShowTenant` (`ShowTenant.js`): toggle de visibilidad de tenant (probablemente para multi-tenant switching).
- `widget` (`widgetSlice.js`): lista de widgets colocados en el PDF, sus posiciones y propiedades (el estado "local" de un documento en edición que puede persistirse o descartarse).
- `sidebar` (`sidebarReducer.js`): estado del sidebar (abierto/cerrado, ítem seleccionado).

El store se configura en `apps/OpenSign/src/redux/store.js` usando `configureStore` de Redux Toolkit.

Context API (`apps/OpenSign/src/context/`)

Dos contextos especializados:

- `GuidelinesContext` (`GuidelinesContext.jsx`): gestiona referencias DOM a las guías de alineación mostradas al arrastrar/redimensionar widgets sobre el PDF. Mantiene refs por página (`guideRefs`, `activePageRef`) y refs globales del canvas (`canvasGuideRefs`) — evita re-renders de todo el árbol en cada movimiento de píxel, usando directamente DOM manipulation a través de refs.
- `ScrollPdfContext` (`ScrollPdfContext.jsx`): probablemente rastrea la posición de scroll del PDF (tema oscuro/claro también mencionado en sesiones previas, manejado vía `useIsDarkTheme` hook + `data-theme` en `html`).

Custom hooks (`apps/OpenSign/src/hook/`)

- `useDraggable` — detecta drag-and-drop en widgets, calcula deltas de movimiento.
- `useElSize` — obtiene ancho/alto de elementos DOM.
- `useIsDarkTheme` — consulta `[data-theme]` en el DOM para activar/desactivar tema oscuro; usado por `ThemeToggle.jsx`.
- `useManifestUrl` — resuelve URLs desde el manifest (probablemente para PWA o assets dinámicos).
- `usePdfPinchZoom` — zoom con pinch gesture en touch devices; mantiene refs de escala y punto de origen.
- `useScript` — carga scripts externos de forma lazy.
- `useUserListFilters` — estado de filtros/búsqueda en la lista de usuarios (`UserList.jsx`).
- `useWidgetDrag` — orquesta drag-and-drop de widgets sobre el lienzo del PDF.
- `useWindowSize` — reacciona a cambios de viewport (responsive).

Componentes por dominio

`apps/OpenSign/src/components/` organiza componentes por feature, no por tipo técnico (buttons, modals, etc.):

- `pdf/` (40+ componentes) — editor PDF: renderizado de páginas (`RenderPdf`, `RenderAllPdfPage`), widget placement (`Placeholder`, `WidgetComponent`, `WidgetList`), edición de template (`EditTemplate`), destinatarios (`RecipientList`, `SignersInput`), tools (`PdfHeader`, `PdfTools`), etc. Documentado en profundidad en `docs/document-preparation-and-sending.md`.
- `emailbuilder/` — constructor visual de emails; usa `@usewaypoint/email-builder`.
- `emaileditor/` — editor de cuerpos de email con Quill.
- `opensigndrive/` — gestor de documentos (drive/almacenamiento).
- `bulksend/` — envío masivo de documentos.
- `dashboard/` — vistas del dashboard.
- `auth/` — componentes de autenticación (login, register, etc.).
- `users/` — lista y gestión de usuarios.
- `preferences/` — preferencias de usuario.
- `sidebar/` — menú lateral.
- `shared/` — componentes reutilizables (campos comunes, modales genéricas).
- `__tests__` — tests unitarios de componentes.

Componentes raíz (`Header.jsx`, `Footer.jsx`, `Title.jsx`, `ThemeToggle.jsx`, `DragProvider.jsx`) actúan como wrappers y proveedores de contexto globales.

Primitivos (24 componentes en `apps/OpenSign/src/primitives/`)

Componentes UI de bajo nivel y modales reutilizables:

- **Guards de ruta:** `ValidateRoute.jsx`, `Validate.jsx`, `LazyPage.jsx`.
- **Modales:** `SessionExpiredModal.jsx`, `AddContact.jsx`, `DeleteUserModal.jsx`, `PasswordResetModal.jsx`, `PdfDeclineModal.jsx`, `LinkUserModal.jsx`, `ModalUi.jsx`, `DateWidgetModal.jsx`.
- **Utilidades:** `Alert.jsx`, `CheckCircle.jsx`, `Icon.jsx` (librería de iconos), `Loader.jsx`, `LoaderWithMsg.jsx`, `Tooltip.jsx`, `Tour.jsx`, `TourContentWithBtn.jsx`.
- **Render:** `RenderReportCell.jsx`, `SignerCell.jsx`, `DownloadPdfZip.jsx`, `DotLottieReact.jsx`, `HandleError.jsx`.

Estilos — Tailwind + DaisyUI + MUI + Radix

El proyecto combina cuatro sistemas de estilos:

1. **Tailwind CSS** (`tailwind.config.js`): framework base, con colores personalizados (`gray`, `success-scale`, `error-scale`, `warning-scale`, `blue-light`, `royal`, `night`, `peach`, `pink`, `purple`), font family `Nunito`, y redondeo consistente.
2. **DaisyUI** (`daisyui` plugin en Tailwind): componentes UI listos (buttons, modals, cards, etc.) con prefijo personalizado `op-` (OpenSign prefix) para evitar conflictos con clases globales. Define dos temas:
 - `opensigndark`: fondo `#101828` (gray-900), texto `#e6e8eb`, acentos azul/blanco.
 - `opensigncss`: fondo claro `#edf4fc`, azul rey `#1d4ed8` como primario, texto negro.
3. **Material UI** (`@mui/material`, `@mui/icons-material`): componentes como Tables, DatePickers, Icons. Coexiste sin conflictos de namespacing.
4. **Radix UI Themes** (`@radix-ui/themes`): system de temas agnóstico, probablemente usado en componentes específicos de preferencias o accesibilidad.

El tema se controla mediante `data-theme` en el `<html>` (ej. `<html data-theme="opensigndark">`). El hook `useIsDarkTheme` consulta ese atributo; el componente `ThemeToggle.jsx` permite cambiar entre temas y persiste la elección en `localStorage.userSettings`.

Estilos específicos viven en `apps/OpenSign/src/styles/`:

- `dark-theme-improvements.css` — refinamientos de dark mode (overrides de colors).
- `opensigndrive.css` — estilos del módulo de drive.
- `quill.css` — overrides de Quill editor.
- `signature.css` — estilos de canvas de firma.
- `widgetNameModal.css` — estilos modales de widgets.

Layout (`apps/OpenSign/src/layout/`)

- `HomeLayout.jsx` — envuelve todas las rutas autenticadas; renderiza `Header`, `Sidebar`, y `<Outlet />` para las sub-rutas. Si `sessionStatus` es `false` en Redux, muestra `SessionExpiredModal` en lugar del contenido.

Páginas (22 archivos en `apps/OpenSign/src/pages/`)

Rutas principales de la aplicación. Ejemplos representativos:

- **Login.jsx** — formulario de login con contraseña; valida sesión Parse, resuelve `contracts_Users` y redirige según rol.
- **GuestLogin.jsx** — login OTP para firmantes invitados; maneja verificación de email y creación de contacto.
- **Form.jsx** (66 KB) — editor de PDF/template; arrastra widgets, asigna signatarios, configura orden de firma y OTP.
- **TemplatePlaceholder.jsx** (91 KB) — similar a `Form` pero para editar plantillas guardadas.
- **PlaceHolderSign.jsx** (95 KB) — ubicación de campos por parte del firmante (orden 2 en la ceremonia de firma).
- **PdfRequestFiles.jsx** (94 KB) — firma del documento (ceremonia de firma completa).
- **Opensigndrive.jsx** (36 KB) — explorador de documentos/templates guardados.
- **Report.jsx** — vista de reporte de un documento (auditoría de firma).
- **Dashboard.jsx** — resumen de documentos y actividad reciente.
- **UserProfile.jsx** — editar datos del usuario extendido (`contracts_Users`).
- **UserList.jsx** (32 KB) — administración de usuarios de la organización.
- **Managesign.jsx** (22 KB) — gestión de perfiles/tipos de firma.
- **Preferences.jsx** (23 KB) — configuración de usuario (idioma, timezone, notificaciones).
- **VerifyDocument.jsx** — verificar integridad de un documento firmado.
- **EmailBuilder.tsx** — constructor visual de emails (usando `@usewaypoint`).
- **SignyourselfPdf.jsx** (59 KB) — caso especial: autofirma de documento.
- Páginas de soporte: `ChangePassword` , `ForgetPassword` , `DebugPdf` , `AddAdmin` , `UpdateExistUserAdmin` , `DocSuccessPage` , `PageNotFound` .

Utils ([apps/OpenSign/src/utils/](#))

Utilidades transversales:

- **lazyWithRetry** — detección y handling de errores de código codeado durante deploy.
- **withSessionValidation** (usado en 20+ componentes) — HOF que valida `localStorage.TenantId` y `Parse.User.current().getSessionToken()` antes de ejecutar un handler. Si falla, despacha `sessionStatus(false)` y aborta.
- **fileUtils** — validación de archivos, conversión de formatos.
- **widgetUtils** — cálculo de posiciones, tamaños y escalas de widgets.
- **prefillUtils** — datos precompletados en widgets.
- **notificationManager** — centraliza el manejo de toasts (usando `sonner`).
- **sanitizeFileName** — limpia nombres de archivo.
- **upgradeProgress** — UI de progreso de actualización de versión.

Otros directorios

- **constant/** — configuraciones: `Utils.js` (165 KB, funciones de utilidad diversas como `defaultWidthHeight` , `replaceMailVariables`), `appinfo.js` (URLs de backend según env), `const.js` , y getters de hash/query.
- **json/** — data estática: `FormJson.js` , `ReportJson.js` , `dashboardJson.js` , `menuJson.js` , `BulkSendSteps.jsx` .
- **reports/** — vistas de reporte: `contact/` , `document/` , `template/` .
- **assets/** — imágenes, iconos SVG, locales i18n.

Diagrama — flujo de componentes y capas

```
graph TD
    A["App.jsx<br/>Router + Lazy Loading"] --> B["ValidateRoute<br/>Validate<br/>Rutas públicas"]
    A --> C["HomeLayout<br/>Header + Sidebar"]
    B --> D["Pages<br/>Login, GuestLogin, Form"]
    C --> E["Pages<br/>Dashboard, Profile, Drive"]

    E --> F["Componentes por Feature"]
    F --> F1["pdf/<br/>RenderPdf, Placeholder, EditTemplate"]
    F --> F2["emailbuilder/<br/>EmailBuilder, CustomizeMail"]
    F --> F3["opensigndrive/<br/>Drive explorer"]
    F --> F4["auth/, users/, preferences/"]

    F1 --> G["Redux Store"]
    F --> H["Context<br/>GuidelinesContext, ScrollPdfContext"]

    G --> G1["user reducer<br/>sessionStatus, userInfo"]
    G --> G2["widget reducer<br/>lista de widgets"]
    G --> G3["sidebar, appInfo, TourSteps"]

    F1 --> I["Custom Hooks<br/>useDraggable, useElSize, usePdfPinchZoom"]
    F1 --> J["Primitives<br/>Modales, Guards, Utilities"]

    F --> K["Estilos"]
    K --> K1["Tailwind + DaisyUI<br/>prefijo op-, temas dark/light"]
    K --> K2["MUI + Radix UI<br/>componentes específicos"]
    K --> K3["CSS custom<br/>quill.css, opensigndrive.css"]

    L["Utils<br/>lazyWithRetry, withSessionValidation, widgetUtils"] --> F
    L --> D
```

Convenciones

- **Naming de archivos:** componentes en PascalCase (`Header.jsx`), hooks en camelCase con prefijo `use` (`useIsDarkTheme.js`), utilities en camelCase (`fileUtils.js`).
- **Imágenes e íconos:** localizados en `assets/` , con un wrapper SVG/Lottie en `primitives/Icon.jsx` .
- **Traducciones:** via `react-i18next` , con loader de backend (`i18next-http-backend`).
- **Modales y dialogs:** centralizados en `primitives/` , reutilizados desde múltiples páginas.
- **Drag & Drop:** orquestado por `GuidelinesContext` (refs de alignment), `useDraggable` y `useWidgetDrag` (detectores), y componentes como `Placeholder` (receptores).

Puntos de extensión

1. **Nueva página:** crear archivo en `pages/` , envolver en `lazyWithRetry` , agregar ruta en `App.jsx` .
2. **Nuevo widget de PDF:** agregar tipo a la lista `draggableItems` en `WidgetComponent.jsx` , default de tamaño en `constant/Utils.js` , render en `getWidgetType.jsx` .
3. **Nuevo estado global:** agregar reducir a `redux/reducers/` y agregarlo al store en `redux/store.js` .
4. **Nuevo contexto de UI:** crear proveedor en `context/` , envolver las rutas relevantes en `HomeLayout.jsx` .
5. **Nuevos estilos de tema:** extender `tailwind.config.js` (colores/tipografía) o agregar tema DaisyUI en la sección `daisyui.themes` .

Catálogo de Cloud Functions

Este documento cataloga las **53 Parse Cloud Functions** registradas vía `Parse.Cloud.define` en `apps/OpenSignServer/cloud/main.js`, los **6 endpoints REST** montados por fuera de Parse en `apps/OpenSignServer/cloud/customRoute/customApp.js` (Express puro), y los **11 hooks automáticos** (`beforeSave` / `afterSave` / `afterFind`) registrados sobre clases Parse. Cada entrada fue verificada contra el archivo real en `cloud/parsefunction/` o `cloud/customRoute/` — parámetros, retorno, errores y permisos reflejan el código tal como está hoy, no la documentación previa de OpenSign upstream. Las funciones están agrupadas en secciones temáticas que siguen la organización real del código.

Autenticación y administración de usuarios

usersignup

Archivo: `cloud/parsefunction/usersignup.js` **Parámetros:** `request.params.userDetails` (objeto: `email`, `password`, `phone`, `company`, `role`, `name`, `pincode`, `country`, `state`, `city`, `address`, `jobTitle`, `timezone`) **Retorno:** `{ message: 'User sign up', sessionToken }`, o `{ message: 'User already exist' }` si ya existe un registro `<rol>_Users` para ese usuario **Errores:** relanza lo que arroje `createUserAccount` (p.ej. `Parse.Error.USERNAME_TAKEN` si el email ya existe) **Permisos:** público (no requiere `request.user`); usa `useMasterKey` para crear el tenant (`partners_Tenant`) y el registro extendido de usuario.

SendOTPMailV1

Archivo: `cloud/parsefunction/SendMailOTPv1.js` **Parámetros:** `email`, `TenantId` (opcional), `docId` (opcional) **Retorno:** `'Otp send'`, o `'Please Enter valid email'` si falta `email` **Errores:** no lanza `Parse.Error`; el catch interno hace `console.log` y devuelve el objeto de error crudo **Permisos:** público; `useMasterKey` para leer/crear el registro `defaultdata_Otp`.

AuthLoginAsMail

Archivo: `cloud/parsefunction/AuthLoginAsMail.js` **Parámetros:** `otp`, `email` **Retorno:** payload de sesión devuelto por `/loginAs` (login sin password), o strings `'Invalid Otp'` / `'user not found!'` / `'Result not found'` **Errores:** no lanza; retorna mensajes de string en cada rama de fallo; bloquea el OTP tras `MAX_OTP_ATTEMPTS = 5` intentos fallidos **Permisos:** público; llama a `/loginAs` con `X-Parse-Master-Key`; `useMasterKey` en las queries de OTP/usuario.

getUserDetails

Archivo: `cloud/parsefunction/getUserDetails.js` **Parámetros:** `email` (opcional), `userId` (opcional) **Retorno:** registro `contracts_Users` completo; o `{ exists: true|false }` si se pasó `email`; `''` si no hay resultado **Errores:** `Parse.Error.INVALID_SESSION_TOKEN` si no hay `email` ni `request.user`; `Parse.Error(code, msg)` genérico en catch **Permisos:** requiere `email` en params o `request.user`; `useMasterKey`.

verifyemail

Archivo: `cloud/parsefunction/VerifyEmail.js` **Parámetros:** `otp`, `email` **Retorno:** `{ message: 'Email is already verified.' }` o `{ message: 'Email is verified.' }` **Errores:** `Parse.Error.INVALID_SESSION_TOKEN` si no hay `request.user`; error código 400 `'OTP is invalid.'` o `'Something went wrong...'` **Permisos:** requiere `request.user`; `useMasterKey` para validar OTP y guardar `emailVerified`.

isextenduser

Archivo: `cloud/parsefunction/isextenduser.js` **Parámetros:** `email` **Retorno:** `{ isUserExist: true|false }` **Errores:** `Parse.Error(code, message)` genérico en catch **Permisos:** público; `useMasterKey`.

addadmin

Archivo: `cloud/parsefunction/AddAdmin.js` **Parámetros:** `userDetails` (mismo shape que `usersignup`) **Retorno:** `{ message: 'User already exist' }` o `{ message: 'User sign up', sessionToken }` **Errores:** el catch externo solo hace `console.log` — **no relanza** el error (fallo silencioso) **Permisos:** público (no valida `request.user`); `useMasterKey`; crea `contracts_Organizations` y `contracts_Teams` ("All Users") y fija rol `contracts_Admin` vía `addTeamAndOrg`.

checkadminexist

Archivo: cloud/parsefunction/CheckAdminExist.js **Parámetros:** ninguno **Retorno:** 'exist' o 'not_exist' **Errores:** Parse.Error(code, msg) genérico **Permisos:** público; useMasterKey .

updateuserasadmin

Archivo: cloud/parsefunction/UpdateExistUserAsAdmin.js **Parámetros:** email, masterkey **Retorno:** 'admin_created' **Errores:** Parse.Error(404, 'Invalid master key.'), Parse.Error.DUPLICATE_VALUE (ya existe admin), Parse.Error.OBJECT_NOT_FOUND, Parse.Error(code, msg) genérico **Permisos:** gate por secreto compartido (masterkey == process.env.MASTER_KEY), no por request.user ; useMasterKey .

getuserlistbyorg

Archivo: cloud/parsefunction/getUserListByOrg.js **Parámetros:** organizationId **Retorno:** array de contracts_Users (JSON) de la organización **Errores:** INVALID_SESSION_TOKEN, INVALID_QUERY, OBJECT_NOT_FOUND, OPERATION_FORBIDDEN **Permisos:** requiere request.user ; valida server-side que la org solicitada pertenezca al tenant del caller y que un no-admin solo liste su propia organización; useMasterKey .

loginuser

Archivo: cloud/parsefunction/loginUser.js **Parámetros:** email, password **Retorno:** JSON del usuario devuelto por Parse.User.logIn **Errores:** Parse.Error.OBJECT_NOT_FOUND, Parse.Error.PASSWORD_MISSING ; relanza errores de Parse.User.logIn (credenciales inválidas, etc.) **Permisos:** público; usa Parse.User.logIn (no master key).

adduser

Archivo: cloud/parsefunction/addUser.js **Parámetros:** phone, name, password, organization ({objectId, company}), team, tenantId, timezone, role, email **Retorno:** JSON del nuevo contracts_Users **Errores:** INVALID_SESSION_TOKEN, OBJECT_NOT_FOUND, OPERATION_FORBIDDEN (múltiples checks de autorización), INVALID_QUERY, 400 genérico **Permisos:** requiere request.user con rol contracts_Admin / contracts_OrgAdmin ; fuerza tenantId al del caller; role limitado a OrgAdmin / Editor / User (nunca Admin); useMasterKey para lookups, ACL explícita en el nuevo registro.

senddeleterequst

Archivo: cloud/parsefunction/sendDeleteUserMail.js **Parámetros:** userId, app (opcional) **Retorno:** 'mail sent.' **Errores:** INVALID_SESSION_TOKEN, INVALID_QUERY, SCRIPT_FAILED (target no es admin, o falla el envío) **Permisos:** requiere request.user ; solo permite la acción si el usuario objetivo tiene rol contracts_Admin ; useMasterKey .

resetpassword

Archivo: cloud/parsefunction/resetPassword.js **Parámetros:** userId, password **Retorno:** { status: 'success', message: 'Password has been reset.' } **Errores:** INVALID_QUERY (params faltantes o intento de resetear la propia contraseña), INVALID_SESSION_TOKEN, OBJECT_NOT_FOUND **Permisos:** requiere request.user con rol contracts_Admin / contracts_OrgAdmin del mismo tenant; el target no puede ser Admin ni el propio caller; useMasterKey .

Documentos

signPdf

Archivo: cloud/parsefunction/pdf/PDF.js **Parámetros:** docId, userId (signer/contacto, opcional), isCustomCompletionMail, mailProvider, signature, pdfFile (base64), header x-real-ip, header public_url **Retorno:** { status: 'success', data: <imageUrl del PDF firmado> } **Errores:** OBJECT_NOT_FOUND ('Document not found.'), INVALID_SESSION_TOKEN (si el doc tiene OTP habilitado y no hay request.user), OPERATION_FORBIDDEN (orden estricto de firma violado), error 400 ('Pdf file not present!' / 'Please provide required parameters!') **Permisos:** autenticación condicional a IsEnableOTP del documento; firma digitalmente con el PFX del tenant o el de process.env.PFX_BASE64 ; useMasterKey en todo el flujo.

getDocument

Archivo: cloud/parsefunction/getDocument.js **Parámetros:** docId, include (opcional), header sessiontoken **Retorno:** JSON del documento (sin TenantId.FileAdapters / PfxFile) **Errores:** no lanza — devuelve objetos { error: '...' } en casi todos los casos de fallo (documento no encontrado, sin acceso, sesión inválida) **Permisos:** si IsEnableOTP es true, exige sessiontoken válido y chequea ACL de lectura; si no, acceso por docId; useMasterKey para la query.

getDrive

Archivo: cloud/parsefunction/getDrive.js **Parámetros:** limit, skip, docId (opcional, para listar contenido de una carpeta), header sessiontoken **Retorno:** array de contracts_Document (excluye AuditTrail / OriginalDocument / SignedDocument) **Errores:** no lanza — devuelve { error: '...' } en fallos **Permisos:** requiere sessiontoken válido resuelto vía /users/me; useMasterKey para la query, filtrada por CreatedBy.

batchdocuments

Archivo: cloud/parsefunction/createBatchDocs.js **Parámetros:** Documents (JSON string), headers sessiontoken, type (quicksend / bulksend), x-real-ip, origin / public_url **Retorno:** { total, created, failed } **Errores:** INVALID_SESSION_TOKEN si no hay request.user; Parse.Error(code, msg) genérico **Permisos:** requiere request.user; hace POST /batch contra el propio Parse Server con el sessionToken del caller; envía email de resumen al dueño al finalizar.

linkcontacttodoc

Archivo: cloud/parsefunction/linkContactToDoc.js **Parámetros:** email, docId, name, phone, jobTitle, company **Retorno:** { contactId } **Errores:** OBJECT_NOT_FOUND (documento o usuario no encontrado), OPERATION_FORBIDDEN ('unauthorized' si el email no está en los Placeholders del documento) **Permisos:** sin chequeo explícito de request.user (autoriza por coincidencia email/placeholder); useMasterKey; otorga ACL de lectura/escritura sobre el documento al nuevo usuario/contacto.

declinedoc

Archivo: cloud/parsefunction/declinedocument.js **Parámetros:** docId, reason (opcional), userId (solo si no hay request.user), header public_url **Retorno:** 'document declined' / 'document already declined' **Errores:** SCRIPT_FAILED (falta docId), OPERATION_FORBIDDEN (caller no es creador ni firmante), OBJECT_NOT_FOUND, INVALID_SESSION_TOKEN (solo si el documento tiene OTP habilitado) **Permisos:** el caller debe ser el creador o figurar en Signers; useMasterKey.

getsigners

Archivo: cloud/parsefunction/getSigners.js **Parámetros:** search (opcional, sobre Name/Email) **Retorno:** array de contracts_Contactbook creados por el caller **Errores:** INVALID_SESSION_TOKEN si no hay request.user; relanza errores de query **Permisos:** requiere request.user; la query corre con el sessionToken del caller (no master key).

savefile

Archivo: cloud/parsefunction/saveFile.js **Parámetros:** fileBase64, id (opcional), fileName **Retorno:** { url } **Errores:** INVALID_QUERY (falta archivo), OBJECT_NOT_FOUND (usuario no encontrado), INVALID_SESSION_TOKEN, 400 genérico en fallo de subida **Permisos:** requiere request.user; useMasterKey para resolver el usuario extendido; aplanar PDFs (flattenPdf) antes de subir.

filterdocs

Archivo: cloud/parsefunction/filterDocs.js **Parámetros:** searchTerm, limit, skip, caseSensitive **Retorno:** array de contracts_Document que matchean por Name **Errores:** INVALID_SESSION_TOKEN, INVALID_PARAMETER (searchTerm no es string), SCRIPT_FAILED (fallo de query) **Permisos:** requiere request.user; query acotada a CreatedBy = request.user; useMasterKey.

createduplicate

Archivo: cloud/parsefunction/createDuplicate.js **Parámetros:** templateId **Retorno:** JSON del nuevo contracts_Template duplicado **Errores:** INVALID_SESSION_TOKEN, INVALID_QUERY ('You cannot create duplicate of this template.') **Permisos:** requiere request.user; el template origen debe pertenecer al caller (CreatedBy); useMasterKey.

generatecertificate

Archivo: cloud/parsefunction/generateCertificatebydocId.js **Parámetros:** docId **Retorno:** { CertificateUrl } (vacío si el documento no está completo o ya tiene certificado) **Errores:** INVALID_QUERY (falta docId), Parse.Error(code, message) genérico **Permisos:** sin chequeo explícito de request.user; useMasterKey; firma digitalmente el certificado con el PFX de process.env.PFX_BASE64.

fileupload

Archivo: cloud/parsefunction/fileUpload.js **Parámetros:** url **Retorno:** { url: <url firmada con JWT, expira en 200s> } **Errores:** relanza cualquier error capturado **Permisos:** sin auth propia; el JWT se firma con process.env.MASTER_KEY como secreto.

forwarddoc

Archivo: cloud/parsefunction/ForwardDoc.js **Parámetros:** docId, recipients (array, máx. 10) **Retorno:** respuesta del último envío de sendMailWithAttachment **Errores:** INVALID_SESSION_TOKEN, INVALID_QUERY (recipients faltante/inválido), OBJECT_NOT_FOUND, 400 genérico en fallo de envío **Permisos:** requiere request.user; el documento debe pertenecer al caller (CreatedBy), no archivado ni rechazado; useMasterKey.

recreatedoc

Archivo: cloud/parsefunction/recreateDocument.js **Parámetros:** docId **Retorno:** { objectId, createdAt, updatedAt } del documento nuevo **Errores:** INVALID_QUERY (falta docId), INVALID_SESSION_TOKEN (no autenticado, o documento IsSignyourself), OBJECT_NOT_FOUND **Permisos:** requiere request.user; useMasterKey; limpia ACL/audit/decline y resetea el valor de los widgets de los placeholders.

createdocumentfromapp

Archivo: cloud/parsefunction/createDocumentFromApp.js **Parámetros:** document (objeto con Name, URL, ExtUserPtr, CreatedBy, Signers, Placeholders, SignatureType, Bcc, Cc, PenColors, etc.) **Retorno:** el contracts_Document creado **Errores:** INVALID_JSON (falta el payload document), INVALID_SESSION_TOKEN **Permisos:** requiere request.user; useMasterKey; actualiza el contador de documentos del ExtUserPtr.

Plantillas

getTemplate

Archivo: cloud/parsefunction/GetTemplate.js **Parámetros:** templateId, header sessiontoken **Retorno:** JSON del template (sin TenantId.FileAdapters / PfxFile), o { error: '...' } **Errores:** no lanza — devuelve { error: "template deleted or you don't have access." } / { error: 'Invalid session token' } **Permisos:** resuelve el email del caller vía sessiontoken (/users/me); visibilidad acotada a dueño, equipos (SharedWith) o usuarios compartidos (SharedWithUsers); useMasterKey.

saveastemplate

Archivo: cloud/parsefunction/saveAsTemplate.js **Parámetros:** docId **Retorno:** el contracts_Template guardado **Errores:** INVALID_SESSION_TOKEN, OBJECT_NOT_FOUND **Permisos:** requiere request.user; el documento origen debe pertenecer al caller; useMasterKey.

Firmas

savesignature

Archivo: cloud/parsefunction/saveSignature.js **Parámetros:** signature, userId, initials, id (opcional, para update), title, stamp **Retorno:** el contracts_Signature guardado **Errores:** INVALID_QUERY (falta userId, o userId !== request.user.id) **Permisos:** userId debe coincidir con request.user.id; useMasterKey.

managesign

Archivo: cloud/parsefunction/manageSign.js **Parámetros:** idénticos a savesignature (implementación casi duplicada) **Retorno:** el contracts_Signature guardado **Errores:** mismos checks INVALID_QUERY que savesignature **Permisos:** userId debe coincidir con request.user.id ; useMasterKey .

getdefaultsignature

Archivo: cloud/parsefunction/getSignature.js **Parámetros:** userId **Retorno:** el contracts_Signature del usuario **Errores:** INVALID_QUERY (falta userId , o no coincide con request.user.id) **Permisos:** userId debe coincidir con request.user.id ; useMasterKey .

updatesignaturetype

Archivo: cloud/parsefunction/updatesignaturetype.js **Parámetros:** SignatureType (array de { name, enabled }) **Retorno:** el contracts_Users actualizado **Errores:** INVALID_SESSION_TOKEN , INVALID_QUERY (ningún tipo habilitado, o solo default habilitado), OPERATION_FORBIDDEN **Permisos:** requiere request.user ; useMasterKey .

Contactos y equipos

getteams

Archivo: cloud/parsefunction/getTeams.js **Parámetros:** active (opcional, filtra IsActive) **Retorno:** array de contracts_Teams de la organización del caller (o []) **Errores:** INVALID_SESSION_TOKEN , Parse.Error(code, msg) genérico **Permisos:** requiere request.user ; useMasterKey .

getcontact

Archivo: cloud/parsefunction/getContact.js **Parámetros:** contactId **Retorno:** el contracts_Contactbook **Errores:** relanza errores de .get() (p.ej. OBJECT_NOT_FOUND) **Permisos:** sin chequeo explícito de request.user ; useMasterKey .

updatecontacttour

Archivo: cloud/parsefunction/updateContactTour.js **Parámetros:** contactId **Retorno:** el contracts_Contactbook actualizado (marca requestSign: true en TourStatus) **Errores:** OBJECT_NOT_FOUND **Permisos:** sin chequeo explícito de request.user ; useMasterKey .

savecontact

Archivo: cloud/parsefunction/savecontact.js **Parámetros:** name, phone, email, tenantId, company, jobTitle **Retorno:** el contracts_Contactbook creado (JSON) **Errores:** DUPLICATE_VALUE ('Contact already exists. '); si no hay request.user la función retorna undefined sin lanzar **Permisos:** requiere request.user (chequeo implícito por if); crea un _User sombra si el email no existe aún; useMasterKey para los saves.

isuserincontactbook

Archivo: cloud/parsefunction/isUserInContactBook.js **Parámetros:** ninguno (usa request.user.email) **Retorno:** el contracts_Contactbook del propio caller, o undefined **Errores:** relanza errores capturados **Permisos:** requiere request.user (chequeo implícito); la query corre con el sessionToken del caller.

updatetourstatus

Archivo: cloud/parsefunction/updateTourStatus.js **Parámetros:** TourStatus, ExtUserId **Retorno:** el contracts_Users actualizado **Errores:** INVALID_SESSION_TOKEN , Parse.Error(code, msg) genérico **Permisos:** requiere request.user ; el .save() se hace sin useMasterKey (depende de ACL/sesión).

createbatchcontact

Archivo: cloud/parsefunction/createBatchContact.js **Parámetros:** contacts (JSON string, array) **Retorno:** { success, failed } (conteos)
Errores: INVALID_SESSION_TOKEN, INVALID_QUERY (falta contacts), INVALID_CONTENT_LENGTH (array vacío), 400 genérico **Permisos:** requiere request.user; usa POST /batch autenticado con X-Parse-Master-Key (no el session token del caller).

editcontact

Archivo: cloud/parsefunction/editContact.js **Parámetros:** contactId, name, email, phone, tenantId, company, jobTitle **Retorno:** el nuevo contracts_Contactbook (JSON) — soft-delete del anterior + creación de uno nuevo **Errores:** INVALID_SESSION_TOKEN, DUPLICATE_VALUE, 400 genérico **Permisos:** requiere request.user; el soft-delete usa el sessionToken del caller, el resto useMasterKey.

gettenant

Archivo: cloud/parsefunction/getTenant.js **Parámetros:** userId (opcional), contactId (opcional) **Retorno:** el partners_Tenant (excluye FileAdapters / PfxFile [/ ContactNumber]) **Errores:** INVALID_SESSION_TOKEN (solo en la rama por userId sin request.user) **Permisos:** la rama por contactId no requiere request.user; la rama por userId sí; useMasterKey.

updatetenant

Archivo: cloud/parsefunction/updateTenant.js **Parámetros:** tenantId, details (CompletionBody / CompletionSubject / RequestBody / RequestSubject / EmailEditorType) **Retorno:** el partners_Tenant actualizado **Errores:** 400 (params faltantes), INVALID_SESSION_TOKEN, OBJECT_NOT_FOUND, OPERATION_FORBIDDEN **Permisos:** requiere request.user con rol contracts_Admin / contracts_OrgAdmin; tenantId debe ser el propio tenant del caller (nunca confía en el valor recibido); useMasterKey.

getlogobydomain

Archivo: cloud/parsefunction/GetLogoByDomain.js **Parámetros:** domain **Retorno:** { logo, favicon, appname, user: 'exist'|'not_exist' } **Errores:** Parse.Error(code, msg) genérico **Permisos:** público (usado antes del login para branding); useMasterKey.

Reportes y utilidades varias

getReport

Archivo: cloud/parsefunction/getReport.js (usa reportsJson.js para las definiciones de reporte y applySearch) **Parámetros:** reportId, limit, skip, searchTerm, signerStatus, header sessiontoken / x-parse-session-token **Retorno:** array de resultados (vía REST de Parse, results) **Errores:** no lanza — devuelve { error: 'Invalid session token' } / { error: 'Report is not available!' } / { error: "You don't have access!" } **Permisos:** resuelve al caller vía sessiontoken; las queries a /classes/... usan X-Parse-Master-Key; el reporte 6TeaPr321t aplica un filtro adicional por equipos/organización del caller.

sendmailv3

Archivo: cloud/parsefunction/sendMailv3.js **Parámetros:** extUserId, from, recipient, subject, text, html, bcc, cc, replyto **Retorno:** { status: 'success' } o { status: 'error' } **Errores:** nunca lanza — todas las excepciones se capturan y se traducen a { status: 'error' } **Permisos:** sin chequeo de request.user; función interna usada por otras cloud functions para despachar mail vía SMTP o Mailgun.

getsignedurl

Archivo: cloud/parsefunction/getSignedUrl.js **Parámetros:** docId (opcional), templateId (opcional), url **Retorno:** URL firmada (presigned S3, o JWT local si useLocal === 'true') **Errores:** INVALID_SESSION_TOKEN (documento con OTP sin auth, o sin docId / templateId y sin autenticar); envuelve otros errores en Parse.Error **Permisos:** autenticación condicional según IsEnableOTP del documento/template, o exigida siempre que no se pase docId / templateId.

updatepreferences

Archivo: cloud/parsefunction/updatePreferences.js **Parámetros:** SignatureType, NotifyOnSignatures, Timezone, UseNameAsSender, SendinOrder, IsTourEnabled, DateFormat, Is12HourTime, IsLTVEabled, DownloadFilenameFormat **Retorno:** el contracts_Users actualizado **Errores:** INVALID_SESSION_TOKEN, INVALID_QUERY (ningún campo válido, o validación de SignatureType), OPERATION_FORBIDDEN **Permisos:** requiere request.user; useMasterKey.

updateemailtemplates

Archivo: cloud/parsefunction/updateEmailTemplates.js **Parámetros:** tenantId, details (CompletionBody / CompletionSubject / RequestBody / RequestSubject / EmailEditorType) **Retorno:** el contracts_Users del caller actualizado (los templates de mail viven en el registro de usuario extendido, no en partners_Tenant, pese al nombre/parámetro) **Errores:** 400 (params faltantes), INVALID_SESSION_TOKEN **Permisos:** requiere request.user; a diferencia de updatetenant, **no valida** que tenantId coincida con el tenant del caller — solo se usa para el include de la query; useMasterKey.

triggerevent

Archivo: cloud/parsefunction/triggerEvent.js **Parámetros:** event, body (con objectId), contactId, header sessiontoken **Retorno:** { message: 'event called!' } / { message: 'User not found!' } / { message: 'Something went wrong!' } **Errores:** nunca propaga una excepción al caller — todo se captura y se traduce a un mensaje **Permisos:** si el documento tiene IsEnableOTP, exige sessiontoken resoluble; si no, sin auth; useMasterKey. Usado para registrar el evento 'viewed' en el AuditTrail.

setwidgetpreferences

Archivo: cloud/parsefunction/setWidgetPreferences.js **Parámetros:** dateWidget ({ isSigningDate, isReadOnly, date, format }) **Retorno:** { WidgetPreferences, updatedAt, createdAt } **Errores:** INVALID_SESSION_TOKEN, INVALID_QUERY (falta dateWidget), OPERATION_FORBIDDEN, Parse.Error genérico **Permisos:** requiere request.user; useMasterKey.

Endpoints REST (customRoute)

Montados en cloud/customRoute/customApp.js como app Express independiente (con su propio cors / express.json), no como Parse.Cloud.define.

POST /docxtopdf

Archivo: cloud/customRoute/docxtopdf.js **Parámetros:** archivo multipart (campo file, solo .docx, límite 50MB vía multer), header sessiontoken **Retorno:** 200 { message: 'success.', url } **Errores:** 400 { error: 'No file uploaded.' }; 403 { error: 'User not linked to tenant.' } / 'Tenant not found for user.'; 400 con mensaje genérico amigable (oculta el error real) con variantes para timeout/tamaño excedido **Permisos:** requiere sessiontoken válido resuelto vía /users/me; las llamadas internas a Parse usan X-Parse-Master-Key; concurrencia limitada (DOCX2PDF_CONCURRENCY, default 1) con timeout de 90s/120s según tamaño.

POST /decryptpdf

Archivo: cloud/customRoute/decryptpdf.js **Parámetros:** archivo multipart en disco (campo file), body.password **Retorno:** 200, stream binario del PDF descifrado (Content-Type: application/pdf) **Errores:** 401 { error: 'Incorrect password.' } en password incorrecta; { error: message } con err.code o 400 en otros casos **Permisos:** sin verificación de sesión ni master key — cualquiera que alcance el endpoint puede intentar descifrar un PDF subido.

GET /delete-account/:userId

Archivo: cloud/customRoute/deleteAccount/deleteUserGet.js **Parámetros:** :userId (path) **Retorno:** 200, página HTML con el formulario de confirmación de borrado por OTP **Errores:** 404 'User not found.' (texto plano) **Permisos:** sin auth — cualquiera con un userId válido puede ver el formulario (el borrado real queda gateado por OTP en el POST).

POST /delete-account/:userId/otp

Archivo: cloud/customRoute/deleteAccount/deleteUserOtp.js **Parámetros:** :userId (path) **Retorno:** 200 { ok: true, cooldownSec, expiresInMin } **Errores:** 404 { error: 'User not found' }; 429 { error: 'Cooldown not finished', retryAfterSec } si se reenvía antes del cooldown; 500 { error: 'Failed to send OTP' } **Permisos:** sin auth; rate-limitado por RESEND_COOLDOWN_SEC (campo DeleteOTPSentAt persistido); el OTP se envía al email registrado del usuario, no a uno arbitrario.

POST /delete-account/:userId

Archivo: cloud/customRoute/deleteAccount/deleteUser.js (deleteUserPost) **Parámetros:** :userId (path), body.otp **Retorno:** 200, texto plano de éxito ('User and all associated data deleted successfully.') u otro mensaje de deleteUser() **Errores:** 404 'Missing userId parameter.' ; 429 (demasiados intentos de OTP inválido, MAX_ATTEMPTS); 400 (OTP faltante/expirado/inválido; cuenta no es admin; aún quedan usuarios del equipo); 500 genérico **Permisos:** sin sesión — depende únicamente del OTP de la cuenta objetivo; solo permitido si el rol de la cuenta es contracts_Admin y no quedan otros usuarios del tenant; cascada de borrado sobre ~11 clases relacionadas vía useMasterKey .

POST /deleteuser/:userId

Archivo: cloud/customRoute/deleteAccount/deleteUser.js (deleteUserByAdmin) **Parámetros:** :userId (path), header sessiontoken **Retorno:** 200 { message: 'User and all associated data deleted successfully.' } **Errores:** 400 { message: 'unauthorized.' } (sin sessiontoken); 400 { message: 'Missing userId parameter.' }; 400 { message: 'User not found.' }; 400 { message: 'You cannot delete your own account.' }; 400 { message: 'Unauthorized.' } (caller no es Admin/OrgAdmin) **Permisos:** requiere sessiontoken válido resuelto a un contracts_Admin / contracts_OrgAdmin; un OrgAdmin queda acotado a su propia organización; useMasterKey para la cascada de borrado.

Hooks automáticos (beforeSave/afterSave/afterFind)

Triggers registrados sobre clases Parse — no son invocables por nombre desde el cliente, corren automáticamente en cada save/find de la clase indicada.

- **afterSave contracts_Document** (cloud/parsefunction/DocumentAftersave.js): en creación, calcula ExpiryDate / OriginIp / NextReminderDate y reescribe el ACL del documento (lectura/escritura para los Signers , o solo para el creador si no hay firmantes). useMasterKey : sí.
- **afterSave contracts_Contactbook** (cloud/parsefunction/ContactBookAftersave.js): en creación sin UserId , crea un _User sombra para el contacto y le asigna ACL; si ya tiene UserId , asegura ACL de lectura/escritura para creador+usuario y marca IsDeleted: false . useMasterKey : sí.
- **afterSave contracts_Template** (cloud/parsefunction/TemplateAfterSave.js): análogo a DocumentAftersave para templates (NextReminderDate , OriginIp , ACL por Signers /creador). useMasterKey : sí.
- **afterSave contracts_Teams** (cloud/parsefunction/TeamsAftersave.js): en creación, agrega el puntero del propio equipo a su array Ancestors (encadenamiento de jerarquía auto-referenciado). useMasterKey : sí.
- **beforeSave contracts_Document** (cloud/parsefunction/DocumentBeforesave.js): valida longitud máxima de Name / Note / Description y el límite de recordatorios (lanza VALIDATION_ERROR / INVALID_QUERY en creación); en update, fija DocSentAt cuando aparece SignedUrl y actualiza el contador de documentos. useMasterKey : no directamente (delega en setDocumentCount).
- **beforeSave contracts_Template** (cloud/parsefunction/TemplateBeforesave.js): mismas validaciones de longitud/recordatorios que Document ; en creación también llama a setTemplateCount . useMasterKey : no directamente.
- **afterFind Parse.User** (cloud/parsefunction/UserAfterFind.js): resuelve ProfilePic a una URL firmada (presigned S3 o JWT local), solo cuando se devuelve un único objeto. useMasterKey : no.
- **afterFind contracts_Document** (cloud/parsefunction/DocumentAfterFind.js): resuelve SignedUrl / URL / CertificateUrl a URLs firmadas y resuelve imágenes de placeholders prefill , solo para un único objeto devuelto. useMasterKey : no.
- **afterFind contracts_Template** (cloud/parsefunction/TemplateAfterFind.js): igual que DocumentAfterFind pero para templates. useMasterKey : no.
- **afterFind contracts_Signature** (cloud/parsefunction/SignatureAfterFind.js): resuelve ImageURL / Initials / Stamp a URLs firmadas para una única firma devuelta. useMasterKey : no.
- **afterFind partners_Tenant** (cloud/parsefunction/TenantAfterFind.js): resuelve Logo / Favicon a URLs firmadas para un único tenant devuelto. useMasterKey : no.

Rate Limiting — FibexSign Backend (OpenSignServer)

Documenta el estado **actual** (post-PR #34, `fix(security)`): flexibilizar rate limiting con niveles diferenciados para navegación y autenticación de `apps/OpenSignServer/utils/authRateLimiter.js`, leído directamente del código en esta sesión — no se reutiliza ningún resumen previo del archivo, dado que su comportamiento cambió respecto a versiones anteriores (de un único bucket estricto a dos niveles diferenciados). Se referencian brevemente `utils/corsOptions.js` y el resto del hardening de `index.js` (`MASTER_KEY_IPS`, `TRUST_PROXY_HOPS`, `ACCOUNT_LOCKOUT_*`) únicamente donde son directamente relevantes al rate limiting.

Dos niveles (buckets)

El middleware `buildAuthRateLimiterMiddleware()` construye **dos limitadores independientes** con `express-rate-limit`, cada uno con su propio `MemoryStore` (en memoria del proceso — no distribuido entre réplicas):

Nivel	Variable de ventana	Variable de máximo	Default máximo	Rutas
Estricto (auth)	<code>AUTH_RATE_LIMIT_WINDOW_MS</code> (default <code>300000</code> = 5 min)	<code>AUTH_RATE_LIMIT_MAX</code>	30 req/ventana/IP+ruta	<code>STRICT_AUTH_PATH_SUFFIXES</code>
Operacional (navegación)	mismo <code>AUTH_RATE_LIMIT_WINDOW_MS</code>	<code>OPERATIONAL_RATE_LIMIT_MAX</code>	300 req/ventana/IP+ruta (o <code>AUTH_RATE_LIMIT_MAX * 10</code> si <code>OPERATIONAL_RATE_LIMIT_MAX</code> no está seteada y <code>AUTH_RATE_LIMIT_MAX</code> si)	<code>OPERATIONAL_PATH_SUFFIXES</code>

Ambos niveles comparten la misma ventana de tiempo (`windowMs`); solo difiere el máximo de requests permitidas y el store en el que se cuentan. Los defaults en `.env.example` (repo root) son:

```
AUTH_RATE_LIMIT_WINDOW_MS=300000
AUTH_RATE_LIMIT_MAX=30
OPERATIONAL_RATE_LIMIT_MAX=300
```

Rutas cubiertas por cada bucket

Definidas por **sufijo** (`request.path.endsWith(suffix)`), no por path exacto — esto las hace robustas al prefijo de montaje de Parse Server (`PARSE_MOUNT`, típicamente `/app`), ya que p. ej. `/app/login.endsWith(/login)` es `true`.

STRICT_AUTH_PATH_SUFFIXES (bucket estricto, `authMax` = 30/5min por default):

- `/login`
- `/requestPasswordReset`
- `/functions/loginuser`
- `/functions/AuthLoginAsMail`
- `/functions/SendOTPMailV1`
- `/functions/addadmin`
- `/functions/usersignup`

OPERATIONAL_PATH_SUFFIXES (bucket operacional, `operationalMax` = 300/5min por default):

- `/functions/getUserDetails`
- `/functions/declinedoc`

`AUTH_RATE_LIMITED_PATH_SUFFIXES` es la unión de ambos arrays y se usa en `isAuthRateLimitedPath()` para decidir si una ruta entra al rate limiting en absoluto; `isStrictAuthPath()` decide cuál de los dos limitadores aplica. Cualquier ruta que no matchee ninguno de los dos arrays de sufijos **no pasa por rate limiting** en este middleware (`return next()` directo).

Nota de nomenclatura: `AUTH_RATE_LIMITED_PATH_SUFFIXES` es un identificador interno del módulo (no exportado); solo `STRICT_AUTH_PATH_SUFFIXES`, `OPERATIONAL_PATH_SUFFIXES`, `isAuthRateLimitedPath`, `isStrictAuthPath`, `buildAuthRateLimiterMiddleware` y `resetAuthRateLimiterStoreForTesting` son exports del archivo.

Cálculo de la clave (key)

Ambos limitadores usan el mismo `keyGenerator`:

```
keyGenerator: request => `${ipKeyGenerator(request.ip)}:${request.path}`
```

Es decir, la clave es **IP + path exacto de la request** (no solo IP). `ipKeyGenerator` es el helper oficial de `express-rate-limit` para normalizar IPv4/IPv6 de forma segura. Esto significa que el límite se cuenta **por combinación IP+ruta**, no de forma agregada entre rutas — un atacante que agote el límite de `/functions/loginuser` desde una IP no afecta su propio límite en `/functions/usersignup` desde la misma IP (cada sufijo de ruta lleva su propio contador dentro del mismo store/nivel).

`request.ip` depende de la configuración de `trust proxy` de Express, fijada en `index.js` vía `TRUST_PROXY_HOPS` (ver más abajo) — si no se configura correctamente detrás de un proxy/load balancer, todas las requests entrantes pueden verse como una sola IP (la del proxy), degradando el aislamiento del rate limiter entre atacante y usuarios legítimos.

Comportamiento al exceder el límite

Al superar `max` dentro de la ventana, `express-rate-limit` responde automáticamente con:

- **Código:** 429 Too Many Requests (default de la librería, no sobrescrito)
- **Cuerpo:** { error: 'Too many requests, please try again later.' } (mismo mensaje para ambos niveles)
- **Headers:** `standardHeaders: true` (headers `RateLimit-*` estándar con el estado del límite) y `legacyHeaders: false` (sin los headers `X-RateLimit-*` legacy)

No hay bloqueo persistente ni backoff exponencial — es una ventana deslizante simple respaldada por `MemoryStore`; al expirar la ventana, el contador para esa clave se resetea.

Testing

`resetAuthRateLimiterStoreForTesting()` limpia ambos `MemoryStore` (`authRateLimiterStore` y `operationalRateLimiterStore`) — pensado para usarse entre tests de integración que ejercitan rutas rate-limited, evitando que el estado de un test contamine el siguiente.

Relación con el resto del hardening

El middleware se monta en `index.js` con `app.use(buildAuthRateLimiterMiddleware())`, **antes** del mount de Parse Server, y **después** de `app.use(cors(buildCorsOptions()))` (`utils/corsOptions.js` — allowlist de orígenes vía `CORS_ALLOWED_ORIGINS`, sin origen — same-origin/curl — siempre permitido). El orden importa: CORS decide si el navegador deja pasar la respuesta al frontend, el rate limiter decide si la request se procesa en absoluto; ambos son independientes entre sí (uno no reemplaza al otro).

Relevante directamente al rate limiter (no se documenta el resto del hardening de `index.js` aquí):

- **TRUST_PROXY_HOPS** (default 0): controla `app.set('trust proxy', N)`. Si es 0, Express no confía en `X-Forwarded-For` y usa la IP de conexión directa como `request.ip` — correcto solo si no hay proxy/load balancer delante. `index.js` emite un `console.warn` explícito si la variable no está seteada, advirtiendo que sin el hop count correcto "the auth rate limiter will not isolate attackers from legitimate users".
- **MASTER_KEY_IPS** (default ['127.0.0.1', ':::1']): restringe desde qué IPs se acepta el `X-Parse-Master-Key` en Parse Server; es un control distinto y complementario al rate limiting (uno limita volumen, el otro restringe origen del master key), pero ambos dependen de que la IP resuelta (`request.ip` / IP real del cliente) sea confiable — lo cual depende a su vez de `TRUST_PROXY_HOPS` bien configurado.
- **ACCOUNT_LOCKOUT_DURATION_MINUTES / ACCOUNT_LOCKOUT_THRESHOLD** (defaults 5 / 5): configuran el `accountLockout` nativo de Parse Server (bloqueo de cuenta tras N intentos fallidos de login), una capa de defensa distinta y adicional al rate limiter por IP — el rate limiter limita volumen de requests hacia el endpoint de login sin importar la cuenta objetivo; el account lockout de Parse Server bloquea una cuenta específica tras fallos repetidos sin importar la IP de origen. Ambos mecanismos se combinan para mitigar credential stuffing / brute force.

Guía de testing

Este documento describe cómo correr y escribir tests en las dos apps del monorepo FibexSign: el frontend `apps/OpenSign` (Vitest) y el backend `apps/OpenSignServer` (Jasmine + nyc + mongodb-runner). También documenta un hallazgo relevante sobre el estado actual de CI: hoy ningún workflow ejecuta la suite de tests automáticamente.

Frontend (`apps/OpenSign`)

Stack y configuración

- Test runner: **Vitest**, configurado en el bloque `test` de `apps/OpenSign/vite.config.js`:

```
test: {
  environment: "jsdom",
  globals: true,
  setupFiles: "../setuptest.js"
}
```

- Librerías de testing: `@testing-library/react`, `@testing-library/user-event`, `@testing-library/jest-dom` (ver `devDependencies` en `apps/OpenSign/package.json`).

Comandos

Desde `apps/OpenSign`:

```
npm run test      # vitest run - corre toda la suite una vez
npm run test:watch # vitest - modo watch
```

Convención de ubicación de tests

Los tests están **co-ubicados** en carpetas `__tests__` junto al código que prueban, no en un directorio de tests centralizado. Ejemplos reales:

- `apps/OpenSign/src/pages/__tests__/Managesign.test.jsx`
- `apps/OpenSign/src/pages/__tests__/Preferences.test.jsx`
- `apps/OpenSign/src/pages/__tests__/UserList.test.jsx`
- `apps/OpenSign/src/components/__tests__/AddUser.test.jsx`
- `apps/OpenSign/src/components/profile/__tests__/PersonalInfoCard.test.jsx`
- `apps/OpenSign/src/hook/__tests__/useIsDarkTheme.test.js`

Patrón: por cada carpeta de código (`pages/`, `components/`, `components/profile/`, `hook/`, etc.) existe una subcarpeta `__tests__` hermana con los tests de esa carpeta, nombrados `<Componente>.test.jsx` (componentes/páginas) o `<hook>.test.js` (hooks/utilidades).

Cómo escribir un test nuevo (ejemplo real)

Para un hook nuevo en `src/hook/`, seguir el patrón de `src/hook/__tests__/useIsDarkTheme.test.js`: crear

`src/hook/__tests__/<miHook>.test.js`, importar el hook con ruta relativa (`../miHook`) y usar `renderHook` / `act` de `@testing-library/react`.

Extracto real del archivo citado:

```
import { act, renderHook } from "@testing-library/react";
import { useIsDarkTheme } from "../useIsDarkTheme";

describe("useIsDarkTheme", () => {
  afterEach(() => {
    setDataTheme(null);
  });

  it("returns false when data-theme is not opensigndark", () => {
    setDataTheme("opensignlight");
    const { result } = renderHook(() => useIsDarkTheme());
    expect(result.current).toBe(false);
  });
  // ...
});
```

Convenciones observadas: `describe / it` de Vitest en modo `globals: true` (no hace falta importarlos explícitamente, aunque `no-external-support-links.test.js` sí los importa desde `"vitest"` — ambos estilos conviven en el repo), nombres de test descriptivos en inglés que explican el comportamiento (`"returns false when ..."`), y `afterEach` para limpiar estado global (aquí, el atributo `data-theme` del `document`).

Tests de invariante de arquitectura (`src/__tests__`)

Además de los tests co-ubicados por feature, existe una carpeta a nivel raíz `apps/OpenSign/src/__tests__` con tests que no prueban un componente puntual sino que actúan como **invariantes de arquitectura ejecutables**: recorren todo `src/` con `fs / path` y verifican una regla global sobre el código fuente, fallando el build de tests si alguien la rompe. Ejemplos reales en el repo:

- `no-blocking-native-dialogs.test.js` — falla si algún archivo fuente (fuera de `__tests__` y `node_modules`) usa `alert( , confirm( , window.alert(` o `window.confirm(`, salvo el propio `src/utils/notificationManager.js` (el único punto autorizado a mostrar diálogos nativos bloqueantes).
- `no-external-support-links.test.js` — verifica que no queden enlaces externos de soporte/comunidad (GitHub, Discord, Twitter, LinkedIn) en `SocialMedia.jsx`, `AddAdmin.jsx`, `UpdateExistUserAdmin.jsx`, que `constant/appinfo.js` no defina `supportEmail`, y que el primitivo `ShareButton.jsx` ni siquiera exista.
- `no-fibesign-brand-text.test.js` — invariante de marca (no dejar texto de marca hardcodeado fuera de lo esperado).
- `utils-circular-dependency.test.js` — invariante de que `src/utils/` no tenga dependencias circulares.

Patrón para agregar una invariante nueva (basado en `no-blocking-native-dialogs.test.js`):

1. Crear `src/__tests__/<regla-en-kebab-case>.test.js`.
2. Resolver `srcRoot` con `path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..")`.
3. Recorrer `srcRoot` recursivamente con `fs.readdirSync(..., { withFileTypes: true })`, excluyendo `__tests__` y `node_modules`, y filtrando por extensión (`.js / .jsx`, excluyendo `*.test.*`).
4. Aplicar una regex/chequeo por archivo y acumular violaciones (patrón `reduce` inmutable, sin mutar arrays).
5. Un único `it` con `expect(violationReport).toBe("")` (o `expect(...).toBe(true/false)` según el caso) que falla con un reporte legible (archivo + línea) cuando hay violaciones.

Este patrón permite codificar reglas de producto/seguridad/marca como tests normales de Vitest, sin herramientas de lint adicionales.

Stack y configuración

- Test runner: **Jasmine**, configurado en `apps/OpenSignServer/spec/support/jasmine.json` :

```
{
  "spec_dir": "spec",
  "spec_files": ["**/*[sS]pec.js"],
  "helpers": ["support/test-env.helper.js", "helper.js"],
  "stopSpecOnExpectationFailure": false,
  "random": false
}
```

- Cobertura: **nyc**.
- Base de datos: **mongodb-runner** levanta un MongoDB efímero para la suite.
- `spec/helper.js` define los hooks globales `beforeAll` / `afterAll` que arrancan y detienen un Parse Server real contra ese Mongo (`startParseServer` , `dropDB` , `stopParseServer` de `spec/utils/test-runner.js`), con un timeout de 2 minutos para el arranque.

Comandos

Desde `apps/OpenSignServer` :

```
npm run test      # mongodb-runner start && TESTING=true jasmine
npm run coverage  # TESTING=true nyc jasmine
npm run lint      # eslint sobre cloud/, index.js y spec/
npm run lint-fix  # eslint --fix sobre los mismos paths
```

`npm run test` levanta primero el Mongo efímero (`mongodb-runner start`) y luego corre Jasmine con `TESTING=true` ; no requiere un Mongo local corriendo de antemano, pero sí requiere Docker o el runtime que `mongodb-runner` necesite para levantar la instancia.

Convención de ubicación y estilo de specs

Todos los specs viven en `apps/OpenSignServer/spec/` , con nombre `*.spec.js` o `*Spec.js` (patrón `**/*[sS]pec.js` de `jasmine.json`). No hay sub-organización por feature dentro de `spec/` : los archivos son planos, nombrados por el área que cubren. Ejemplos reales:

- `spec/Tests.spec.js` — spec de ejemplo/smoke test de Parse Server (llamadas a Cloud Functions y endpoints HTTP básicos).
- `spec/idor-and-enumeration.spec.js` , `spec/otp-security.spec.js` , `spec/server-hardening.spec.js` , `spec/signup-auth-bypass.spec.js` — specs de seguridad, cada uno enfocado en una clase de vulnerabilidad concreta.
- `spec/no-support-email.spec.js` , `spec/brand-name-firma.spec.js` — invariantes de producto/marca, equivalentes en espíritu a los tests de invariante del frontend pero en Jasmine.
- `spec/helper.js` , `spec/support/` — helpers compartidos (arranque/derribo del server de test, utilidades).

Extracto real de `spec/idor-and-enumeration.spec.js` (uso de `describe` / `it` , helpers locales para dar de alta un usuario/tenant, y `Parse.Cloud.run` contra el server de test):

```
describe('getUserDetails unauthenticated enumeration is closed', () => {
  Parse.User.enableUnsafeCurrentUser();

  it('returns only an existence flag, never an objectId, for an existing email with no session', async () => {
    const email = `enum-exists-${Date.now()}@example.com`;
    await signupUser(email, 'Str0ngPassw0rd!');

    const result = await Parse.Cloud.run('getUserDetails', { email });

    expect(result).toEqual({ exists: true });
    expect(Object.prototype.hasOwnProperty.call(result, 'objectId')).toBe(false);
  });
});
```

Convenciones observadas: nombres de `describe` que enuncian la invariante de seguridad que se está cerrando (no solo el nombre de la función), emails únicos con `Date.now()` para evitar colisiones entre corridas, y aserciones que chequean tanto el valor devuelto como la **ausencia** de campos sensibles (`hasOwnProperty`) — relevante para specs anti-IDOR/enumeración.

Cómo escribir un spec nuevo

Seguir el patrón de `idor-and-enumeration.spec.js`: crear `spec/<area>.spec.js`, usar `describe` / `it` de Jasmine, apoyarse en los helpers ya definidos en `spec/helper.js` / `spec/support/` (no reimplementar el arranque de Parse Server), y nombrar el `describe` por la invariante o comportamiento que se protege, no por el nombre técnico de la función bajo test.

CI no ejecuta tests hoy (hallazgo)

El único workflow de CI del repo es `.github/workflows/Docker.yml`. Se dispara en `push` a `main` y `staging`, y su único trabajo (`docker`) hace build multi-arch (amd64/arm64) y push a Docker Hub de las dos imágenes (`opensign/opensign` desde `apps/OpenSign/Dockerhubfile`, `opensign/opensignserver` desde `apps/OpenSignServer/Dockerhubfile`) usando `docker/build-push-action`. En ningún paso del workflow se ejecuta `npm run test`, `vitest` ni `jasmine`.

Esto significa que, en el estado actual del repo, **toda la suite de tests (frontend y backend) corre únicamente de forma local/manual** — no actúa como gate antes de merge a `staging` / `main` ni antes de publicar las imágenes Docker que consume `docker-compose.yml`.

Nota (fuera de alcance de esta documentación): sería valioso agregar un job de test — al menos `npm run test` en `apps/OpenSign` y `npm run test` en `apps/OpenSignServer` — a `Docker.yml` (o a un workflow separado) antes del build/push de imágenes, para que un test roto bloquee la publicación. No se implementa acá; queda como mejora sugerida.

Guía de deployment

Este documento describe cómo desplegar FibexSign con el `docker-compose.yml` de la raíz del repo (el camino soportado para levantar el stack completo: backend, frontend, MongoDB y reverse proxy con TLS), y menciona brevemente las alternativas de deploy standalone por PaaS del backend. Para el detalle completo de variables de entorno, ver [./environment-variables.md](#).

Prerequisitos

- **Docker y Docker Compose** (el `docker-compose.yml` usa la sintaxis de Compose v2, sin declaración de `version:`).
- `make` (el `Makefile` de la raíz orquesta ambos flujos de abajo). En Windows, correr los comandos equivalentes a mano si no hay `make` disponible (ver el contenido del `Makefile` más abajo).
- Un archivo `.env` en la raíz — los targets del `Makefile` lo generan copiando `.env.local_dev`, pero para un deploy real hay que reemplazar sus valores por los de producción (ver checklist más abajo).

`make build` VS `make run`

El `Makefile` de la raíz define dos targets:

```
build:
    @echo "Building with HOST_URL=${HOST_URL}"
    cp .env.local_dev .env
    cd apps/OpenSign && cp ../../.env.local_dev .env && npm install && npm run build
    HOST_URL=${HOST_URL} docker compose up --build --force-recreate

run:
    @echo "Building with HOST_URL=${HOST_URL}"
    cp .env.local_dev .env
    docker compose up -d
```

- `make build`: copia `.env.local_dev` a `.env` (raíz y `apps/OpenSign`), instala dependencias y buildea el frontend **nativamente** (`npm install && npm run build` dentro de `apps/OpenSign`), y luego levanta todo el stack con `docker compose up --build --force-recreate` (reconstruye las imágenes locales y fuerza recreación de contenedores). Usar cuando cambiaste código y necesitas que Compose reconstruya las imágenes desde el `Dockerfile` local en vez de usar las imágenes publicadas en Docker Hub.
- `make run`: copia `.env.local_dev` a `.env` y hace `docker compose up -d`, **sin rebuildear** nada. Usa las imágenes ya existentes (localmente cacheadas o las publicadas en Docker Hub: `opensign/opensignserver:main`, `opensign/opensign:main`). Usar para levantar/reiniciar el stack rápido cuando no cambiaste código.

Ambos targets sobrescriben `.env` con `.env.local_dev` — en un entorno de producción real conviene no usar `make build` / `make run` tal cual, sino preparar el `.env` de producción primero y correr `docker compose up -d` (o `--build` si corresponde) directamente, para no pisarlo con los valores de desarrollo.

Servicios de `docker-compose.yml`

El archivo define 4 servicios en la red `app-network` (bridge):

Servicio	Imagen	Puerto host → contenedor	Depende de	Notas
server	opensign/opensignserver:main	8080:8080	mongo	<code>env_file</code> : <code>.env</code> ; setea <code>NODE_ENV=production</code> , <code>SERVER_URL</code> y <code>PUBLIC_URL</code> derivados de <code>HOST_URL</code> (default <code>https://localhost:3001</code>); volumen <code>opensign-files:/usr/src/app/files</code> para los documentos subidos.
mongo	mongo:latest	27018:27017	—	Puerto host no estándar (27018, no el 27017 default de Mongo); volumen <code>data-volume:/data/db</code> para persistencia.

client	opensign/opensign:main	3000:3000	server	env_file: .env ; es el frontend servido.
caddy	caddy:latest	3001:3001 , 80:80 , 443:443 (TCP y UDP)	—	Reverse proxy con TLS; monta ./Caddyfile , y los volúmenes caddy_data / caddy_config para persistir certificados y estado de Caddy entre reinicios.

Volúmenes nombrados: data-volume (Mongo), web-root (declarado pero sin uso visible en los servicios actuales), caddy_data , caddy_config , opensign-files .

Qué hace el Caddyfile

El Caddyfile de la raíz es corto y define un único sitio:

```
{${HOST_URL}} {
  reverse_proxy client:3000
  handle_path /api/* {
    reverse_proxy server:8080
    rewrite * {uri}
  }
}
```

- El sitio se sirve en el dominio/host dado por la variable de entorno HOST_URL (Caddy hace TLS automático — obtiene y renueva el certificado solo — cuando HOST_URL es un dominio real; con localhost no emite un certificado público, sirve en local).
- Todo el tráfico entra por Caddy y se reparte en dos rutas:
 - Cualquier ruta que **no** empiece con /api/ → proxy al contenedor client (frontend, puerto 3000).
 - Rutas bajo /api/* → handle_path les saca el prefijo /api/ y hace proxy al contenedor server (backend, puerto 8080), reescribiendo la URI resultante.
- En otras palabras: Caddy es el único punto de entrada público (puertos 80/443, más 3001 explícito) y decide, por prefijo de ruta, si la request va al frontend o al backend — el usuario nunca pega directo a los puertos 3000/8080 en un deploy con Caddy.

Checklist de variables de entorno críticas para producción

La lista completa y su documentación viven en [./environment-variables.md](#) . Como mínimo, para un deploy productivo revisar/setear:

- MONGODB_URI — connection string de Mongo (en Compose, apunta a mongo-container:27017 ; en un Mongo gestionado/externo, usar esa URI).
- MASTER_KEY — clave maestra de Parse Server; debe rotarse respecto del valor de ejemplo en .env.local_dev / .env.example .
- SERVER_URL / PUBLIC_URL (y HOST_URL para Compose/Caddy) — deben apuntar al dominio público real, no a localhost .
- Credenciales de storage: DO_ACCESS_KEY_ID , DO_SECRET_ACCESS_KEY , DO_SPACE , DO_ENDPOINT (DigitalOcean Spaces o S3-compatible) para el almacenamiento de documentos.
- Credenciales de email: o bien MAILGUN_API_KEY / MAILGUN_DOMAIN / MAILGUN_SENDER , o bien la config SMTP (SMTP_ENABLE=true , SMTP_HOST , SMTP_PORT , SMTP_USER_EMAIL , SMTP_PASS) — el server no inicializa el envío de mail si falta alguna de las variables requeridas por el modo elegido.
- PFX_BASE64 (y su passphrase asociada) — certificado de firma de documentos en base64; sin esto no funciona la firma digital de PDFs.

Todos estos valores están presentes como placeholders en .env.example / .env.local_dev (raíz) — nunca deployar con esos valores de ejemplo tal cual.

Deploy standalone del backend por PaaS (alternativa a Docker Compose)

Para desplegar **solo** apps/OpenSignServer en un PaaS de un solo servicio (sin el resto del stack — sin client ni caddy gestionados por Compose), el backend trae manifiestos listos:

- `apps/OpenSignServer/app.json` — manifiesto estilo Heroku (define env vars como `PARSE_MOUNT`, `APP_ID`, `MASTER_KEY`, `SERVER_URL`, y el addon `mongolab` para Mongo gestionado).
- `apps/OpenSignServer/openshift.json` — manifiesto para OpenShift.
- `apps/OpenSignServer/scalingo.json` — manifiesto para Scalingo.

Estas opciones son para correr el backend de forma aislada en esas plataformas; no reemplazan al `docker-compose.yml` cuando se necesita el stack completo (frontend + proxy TLS + Mongo) en un solo host.

Troubleshooting básico

- **Puerto de Mongo no estándar (27018):** `docker-compose.yml` mapea `27018:27017`, es decir, Mongo escucha en `27017` **dentro** del contenedor pero se publica en `27018` del host. Si en vez de Docker corrés un `mongod` nativo en el host para desarrollo/debug, tenés que usar `mongodb://localhost:27018/...` para conectarte al Mongo del stack (o `27017` si estás apuntando a un `mongod` nativo separado) — mezclar ambos puertos es la causa más probable de un "no se puede conectar a Mongo" al mezclar Compose con un Mongo local.
- **client o caddy no arrancan / dependencias de orden:** `docker-compose.yml` declara `depends_on` (`server` → `mongo`, `client` → `server`) pero eso solo espera a que el contenedor exista, no a que el proceso adentro esté realmente listo; si `server` tarda en levantar (por ejemplo, esperando a Mongo), `client` / `caddy` pueden arrancar antes de que la API responda — un reinicio del servicio dependiente (`docker compose restart client`) suele bastar en desarrollo.
- **Certificado TLS no se emite:** si `HOST_URL` apunta a `localhost` o a una IP, Caddy no puede emitir un certificado público válido vía ACME; para TLS real, `HOST_URL` debe ser un dominio con DNS público apuntando al host, con los puertos 80/443 accesibles desde internet.
- **.env pisado accidentalmente:** tanto `make build` como `make run` copian `.env.local_dev` sobre `.env` en cada corrida; si editaste `.env` a mano para producción y volvéis a correr `make build` / `make run`, perdés esos cambios. Preparar un `.env` de producción aparte y no usar esos targets del `Makefile` para reapplicarlo, o ajustar el `Makefile` antes de reutilizarlo en ese flujo.